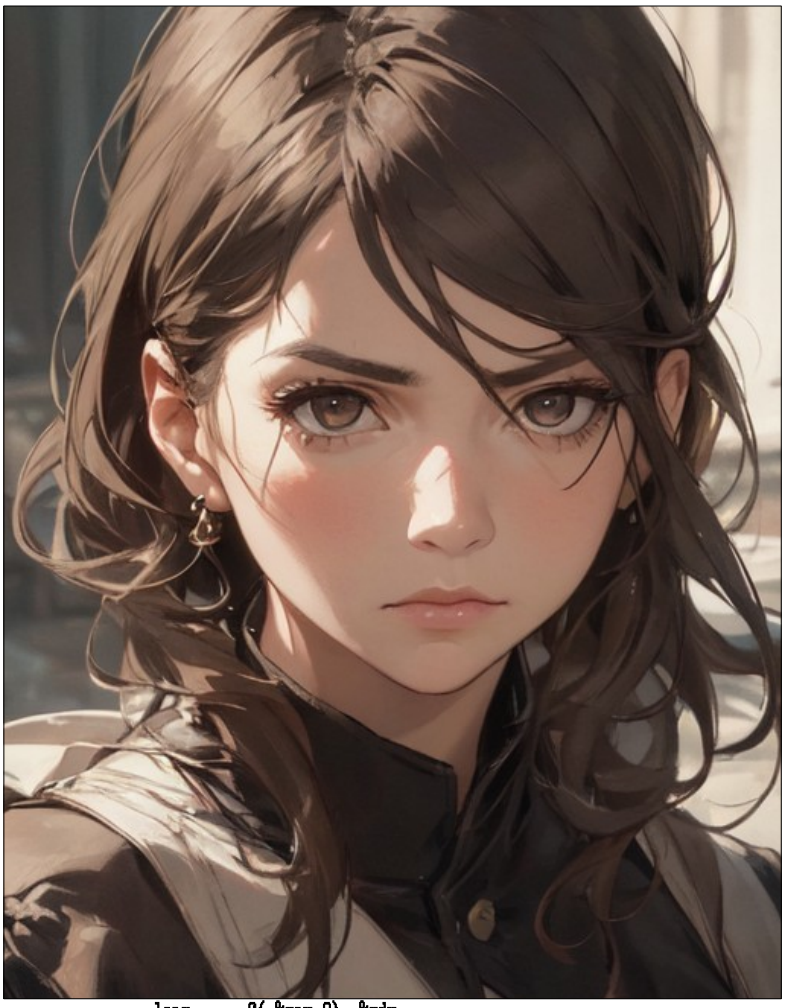
Charlatana

The Fastest UQWORD/uint64_t Quicksort
(AVX2 & AVX512 variants)

https://github.com/Sanmayce/Schmekeriada

Written mostly by Gemini Pro 3.1 AI!



```
// [sanmayce@djuljeto Quicksort_says_v28v29_FullPackage]$ gcc -O3 -mavx2
-std=gnu11 -o benchmark28_SINGLE.asm benchmark28_SINGLE.c -S
// The Assembler code (for the single-threaded) is this (Insertionsort is
embedded/inlined), 640 lines of 256bit Assembly code:
/*
        .p2align 4
        .type   charlatana_v28_recursive, @function
charlatana_v28_recursive:
...
.L115:
        vmovdqu   (%r11), %ymm1
        vpbroadcastq        %xmm4, %ymm0
        addq    $32, %r11
        movslq  %ebx, %rsi
        vpxor   %ymm0, %ymm1, %ymm0
        vpcmpgtq  %ymm0, %ymm2, %ymm3
        vpcmpgtq  %ymm2, %ymm0, %ymm0
        vmovmskpd %ymm3, %rdx
        movq    %rdx, %rax
        salq    $5, %rdx
        vmovdqa   QWORD_PACK_TABLE(%rdx), %ymm3
        popcntl   %eax, %eax
        movq    %rax, %rdx
        salq    $5, %rdx
        vmovdqa   MASK_STORE_TABLE(%rdx), %ymm5
        vpermd    %ymm1, %ymm3, %ymm3
        xorl    %edx, %edx
        vpmaskmovq%ymm3, %ymm5, (%r12)
        leaq    (%r12,%rax,8), %r12
        vmovmskpd %ymm0, %rax
        popcntl   %eax, %edx
        salq    $5, %rax
        addl    %edx, %ebx
        vmovdqa   QWORD_PACK_TABLE(%rax), %ymm0
        movslq  %edx, %rax
        salq    $5, %rax
        vpermd    %ymm1, %ymm0, %ymm1
        vmovdqa   MASK_STORE_TABLE(%rax), %ymm0
        movq    %r11, %rax
        subq    %r12, %rax
        vpmaskmovq%ymm1, %ymm0, 2176(%rsp,%rsi,8)
        sarq    $3, %rax
        cmpl    %eax, %r15d
        jg      .L10
        testl   %r15d, %r15d
        jle     .L10
        movq    %r11, 32(%rsp)
        movl    %r15d, %edx
        movq    %r12, %rdi
        leaq    128(%rsp), %rsi
        vmovdqa   %ymm2, 64(%rsp)
        salq    $3, %rdx
        movslq  %r15d, %r15
        leaq    (%r12,%r15,8), %r12
        vzeroupper
        call    memcpy
        movq    32(%rsp), %r11
        xorl    %r15d, %r15d
        movabsq   $-9223372036854775808, %rax
        vmovdqa   64(%rsp), %ymm2
        vmovq   %rax, %xmm4
        leaq    0(,%rax,8), %rdx
        salq    $5, %rax
        vmovdqa   MASK_STORE_TABLE(%rax), %ymm5
        subq    %rdx, %r13
        vmovmskpd %ymm0, %rax
        xorl    %edx, %edx
        popcntl   %eax, %edx
        vpermd    %ymm1, %ymm3, %ymm3
        salq    $5, %rax
        addl    %edx, %r15d
        vpmaskmovq%ymm3, %ymm5, 0(%r13)
        vmovdqa   QWORD_PACK_TABLE(%rax), %ymm0
        movslq  %edx, %rax
        salq    $5, %rax
        vpermd    %ymm1, %ymm0, %ymm1
        vmovdqa   MASK_STORE_TABLE(%rax), %ymm0
        movq    %r13, %rax
        subq    %r9, %rax
        vpmaskmovq%ymm1, %ymm0, 128(%rsp,%rsi,8)
        sarq    $3, %rax
        cmpl    %eax, %ebx
        jg      .L58
        testl   %ebx, %ebx
        jg      .L116
.L58:
        movq    %r9, %r14
        movq    %r14, %rax
        subq    %r11, %rax
        cmpq    $24, %rax
        jg      .L11
.L114:
...
*/
```

Charlatana_v28.h:

```
#ifndef CHARLATANA_V28_H
#define CHARLATANA_V28_H

#include <immintrin.h>
#include <string.h> // memcpy()
#include <stdint.h>
#include <stddef.h>
#include <omp.h>

// Don't spawn OpenMP tasks for tiny chunks. It chokes the thread queue.
#define OMP_TASK_THRESHOLD 100000
#define INSERTION_THRESHOLD 64

__attribute__((aligned(32))) static const int32_t QWORD_PACK_TABLE[16][8] = {
    {0,1,2,3,4,5,6,7}, {0,1,2,3,4,5,6,7}, {2,3,0,1,4,5,6,7}, {0,1,2,3,4,5,6,7},
    {4,5,0,1,2,3,6,7}, {0,1,4,5,2,3,6,7}, {2,3,4,5,0,1,6,7}, {0,1,2,3,4,5,6,7},
    {6,7,0,1,2,3,4,5}, {0,1,6,7,2,3,4,5}, {2,3,6,7,0,1,4,5}, {0,1,2,3,6,7,4,5},
    {4,5,6,7,0,1,2,3}, {0,1,4,5,6,7,2,3}, {2,3,4,5,6,7,0,1}, {0,1,2,3,4,5,6,7}
};

__attribute__((aligned(32))) static const int64_t MASK_STORE_TABLE[5][4] = {
    {0, 0, 0, 0},
    {-1LL, 0, 0, 0},
    {-1LL, -1LL, 0, 0},
    {-1LL, -1LL, -1LL, 0},
    {-1LL, -1LL, -1LL, -1LL}
};

static inline __m256i Charlatana_cmplt_epu64(const __m256i a, const __m256i b) {
    const __m256i hbit = _mm256_set1_epi64x(0x8000000000000000ULL);
    return _mm256_cmpgt_epi64(_mm256_xor_si256(b, hbit), _mm256_xor_si256(a, hbit));
}

/*
static void charlatana_insertion_sort(uint64_t* arr, int64_t low, int64_t high) {
    for (int64_t i = low + 1; i <= high; i++) {
        uint64_t key = arr[i];
        int64_t j = i - 1;
        while (j >= low && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
*/
static inline void charlatana_insertion_sort(uint64_t* arr, int64_t low, int64_t high) {
    uint64_t* base = arr + low;          /* everything relative to base */
    int64_t   n    = high - low + 1;     /* size (1 when low==high) */

    //if (n <= 1) return;                  /* optional one-liner guard if you prefer */

    /* min to front */
    int64_t min_idx = 0;
    for (int64_t k = 1; k < n; k++) {
        if (base[k] < base[min_idx]) min_idx = k;
    }
    uint64_t tmp = base[0];
    base[0]      = base[min_idx];
    base[min_idx]= tmp;

    /* sentinel insertion */
    for (int64_t i = 1; i < n; i++) {
        uint64_t  key  = base[i];
        uint64_t* jptr = base + i - 1;

        while (*jptr > key) {
            *(jptr + 1) = *jptr;
            jptr--;
        }
        *(jptr + 1) = key;
    }
}

//    if (n > OMP_TASK_THRESHOLD) { should be true on entrance (n = high - low + 1) i.e. for 1..33 n = 32 > 32 is false i.e. for 1..34 n = 33 > 32 is true
static void charlatana_v28_recursive(uint64_t* arr, int64_t low, int64_t high) {
    while (low < high) {
/*
```

```
        if (high - low < 48) {
            charlatana_insertion_sort(arr, low, high);
            return;
        }
*/

/*
        int64_t mid = low + (high - low) / 2;

        uint64_t v1 = arr[low], v2 = arr[mid], v3 = arr[high];
        uint64_t min1 = (v1 < v2) ? v1 : v2;
        uint64_t max1 = (v1 > v2) ? v1 : v2;
        uint64_t min2 = (max1 < v3) ? max1 : v3;
        uint64_t max2 = (max1 > v3) ? max1 : v3;
        uint64_t min3 = (min1 < min2) ? min1 : min2;
        uint64_t max3 = (min1 > min2) ? min1 : min2;
        arr[low] = min3;
        arr[mid] = max3;
        arr[high] = max2;

        uint64_t pivot = arr[mid];
*/

// Masks [
        // === YOUR 25%/50%/75% SAMPLING + BRANCHLESS MEDIAN (perfect for BELL) ===
        // This is exactly what you proposed - spread probes kill the BELL pattern
        // while staying 100 % branchless and only 3 loads + ~12 cheap ops.
        int64_t delta = high - low;
        uint64_t v1   = arr[low   + delta / 4];
        uint64_t v2   = arr[low   + delta / 2];
        uint64_t v3   = arr[high  - delta / 4];

 // min/max of first pair
 uint64_t mask_min_xy = -(uint64_t)(v1 > v2);
 uint64_t mask_max_xy = -(uint64_t)(v1 < v2);
 uint64_t min_xy = v1 ^ ((v1 ^ v2) & mask_min_xy);
 uint64_t max_xy = v1 ^ ((v1 ^ v2) & mask_max_xy);

 // min of (min_xy, v3)
 uint64_t mask_min = -(uint64_t)(min_xy > v3);
 uint64_t overall_min = min_xy ^ ((min_xy ^ v3) & mask_min);

 // max of (max_xy, v3)
 uint64_t mask_max = -(uint64_t)(max_xy < v3);
 uint64_t overall_max = max_xy ^ ((max_xy ^ v3) & mask_max);

 // median = xor of all three minus min minus max
 uint64_t pivot = v1 ^ v2 ^ v3 ^ overall_min ^ overall_max;
// Masks ]

        __m256i v_pivot = _mm256_set1_epi64x(pivot);

        uint64_t* read_ptr_L = arr + low;
        uint64_t* write_ptr_L = arr + low;
        uint64_t* read_ptr_R = arr + high + 1;
        uint64_t* write_ptr_R = arr + high + 1;

        uint64_t buf_L[256];
        uint64_t buf_R[256];
        int count_L = 0;
        int count_R = 0;

        while (read_ptr_R - read_ptr_L >= 4) {
            if (count_L > count_R) {
                __m256i v = _mm256_loadu_si256((__m256i*)read_ptr_L);
                read_ptr_L += 4;

                __m256i m_lt = Charlatana_cmplt_epu64(v, v_pivot);
                int mask_lt = _mm256_movemask_pd(_mm256_castsi256_pd(m_lt));
                int k_lt = __builtin_popcount(mask_lt);
                __m256i perm_lt = _mm256_load_si256((__m256i*)QWORD_PACK_TABLE[mask_lt]);
                __m256i packed_lt = _mm256_permutevar8x32_epi32(v, perm_lt);
                __m256i store_mask_lt = _mm256_load_si256((__m256i*)MASK_STORE_TABLE[k_lt]);
                _mm256_maskstore_epi64((long long*)write_ptr_L, store_mask_lt, packed_lt);
                write_ptr_L += k_lt;

                __m256i m_gt = Charlatana_cmplt_epu64(v_pivot, v);
                int mask_gt = _mm256_movemask_pd(_mm256_castsi256_pd(m_gt));
                int k_gt = __builtin_popcount(mask_gt);
```

```
                __m256i perm_gt = _mm256_load_si256((__m256i*)QWORD_PACK_TABLE[mask_gt]);
                __m256i packed_gt = _mm256_permutevar8x32_epi32(v, perm_gt);
                __m256i store_mask_gt = _mm256_load_si256((__m256i*)MASK_STORE_TABLE[k_gt]);
                _mm256_maskstore_epi64((long long*)(buf_R + count_R), store_mask_gt, packed_gt);
                count_R += k_gt;

                // Branchless [
/*
                // --- USER OPTIMIZATION: Bitwise Branchless Flush ---
                int space_L = (int)(read_ptr_L - write_ptr_L);
                int flush_L = count_L & -(space_L >= count_L); // 0 or count_L

                for (int k = 0; k < flush_L; k++) write_ptr_L[k] = buf_L[k];
                write_ptr_L += flush_L;
                count_L -= flush_L; // Resets cleanly without a branch
*/
                // Branchless ]
                // Branchful [
                int space_L = read_ptr_L - write_ptr_L;
                if (count_L > 0 && space_L >= count_L) {
                    for (int k = 0; k < count_L; k++) write_ptr_L[k] = buf_L[k];
                    write_ptr_L += count_L;
                    count_L = 0;
                }
                // Branchful ]
                // Branchless and for-less [
/*
 ptrdiff_t space_L = read_ptr_L - write_ptr_L;
 int flush_L = count_L & -(int)(space_L >= count_L);

 memcpy(write_ptr_L, buf_L, (size_t)flush_L * sizeof(uint64_t));
 write_ptr_L += flush_L;
 count_L -= flush_L;
*/
                // Branchless and for-less ]
            } else {
                read_ptr_R -= 4;
                __m256i v = _mm256_loadu_si256((__m256i*)read_ptr_R);

                __m256i m_gt = Charlatana_cmplt_epu64(v_pivot, v);
                int mask_gt = _mm256_movemask_pd(_mm256_castsi256_pd(m_gt));
                int k_gt = __builtin_popcount(mask_gt);
                __m256i perm_gt = _mm256_load_si256((__m256i*)QWORD_PACK_TABLE[mask_gt]);
                __m256i packed_gt = _mm256_permutevar8x32_epi32(v, perm_gt);
                __m256i store_mask_gt = _mm256_load_si256((__m256i*)MASK_STORE_TABLE[k_gt]);
                write_ptr_R -= k_gt;
                _mm256_maskstore_epi64((long long*)write_ptr_R, store_mask_gt, packed_gt);

                __m256i m_lt = Charlatana_cmplt_epu64(v, v_pivot);
                int mask_lt = _mm256_movemask_pd(_mm256_castsi256_pd(m_lt));
                int k_lt = __builtin_popcount(mask_lt);
                __m256i perm_lt = _mm256_load_si256((__m256i*)QWORD_PACK_TABLE[mask_lt]);
                __m256i packed_lt = _mm256_permutevar8x32_epi32(v, perm_lt);
                __m256i store_mask_lt = _mm256_load_si256((__m256i*)MASK_STORE_TABLE[k_lt]);
                _mm256_maskstore_epi64((long long*)(buf_L + count_L), store_mask_lt, packed_lt);
                count_L += k_lt;

                // Branchless [
/*
                // --- USER OPTIMIZATION: Bitwise Branchless Flush ---
                int space_R = (int)(write_ptr_R - read_ptr_R);
                int flush_R = count_R & -(space_R >= count_R); // 0 or count_R

                write_ptr_R -= flush_R; // Shift pointer down before loop
                for (int k = 0; k < flush_R; k++) write_ptr_R[k] = buf_R[k];
                count_R -= flush_R;     // Resets cleanly without a branch
*/
                // Branchless ]
                // Branchful [
                int space_R = write_ptr_R - read_ptr_R;
                if (count_R > 0 && space_R >= count_R) {
                    write_ptr_R -= count_R;
                    for (int k = 0; k < count_R; k++) write_ptr_R[k] = buf_R[k];
                    count_R = 0;
                }
                // Branchful ]
                // Branchless and for-less [
/*
 ptrdiff_t space_R = write_ptr_R - read_ptr_R;
```

```
 int flush_R = count_R & -(int)(space_R >= count_R);

 write_ptr_R -= flush_R;
 memcpy(write_ptr_R, buf_R, (size_t)flush_R * sizeof(uint64_t));
 count_R -= flush_R;
*/
                // Branchless and for-less ]
            }
        }

        while (read_ptr_L < read_ptr_R) {
            uint64_t val = *read_ptr_L;
            int is_lt = (val < pivot);
            int is_gt = (val > pivot);

            *write_ptr_L = val;
            write_ptr_L += is_lt;

            buf_R[count_R] = val;
            count_R += is_gt;

            read_ptr_L++;
        }

        for (int k = 0; k < count_L; k++) *write_ptr_L++ = buf_L[k];
        write_ptr_R -= count_R;
        for (int k = 0; k < count_R; k++) write_ptr_R[k] = buf_R[k];

        uint64_t* end_lt = write_ptr_L;
        uint64_t* start_gt = write_ptr_R;

        while (write_ptr_L < write_ptr_R) {
            *write_ptr_L++ = pivot;
        }

        int64_t p_left = (end_lt - arr) - 1;
        int64_t p_right = (start_gt - arr);

        int64_t len_L = p_left - low + 1;
        int64_t len_R = high - p_right + 1;

        if (len_L < len_R) {
            // Right is larger: Task, Recurse, or Sort
            if (len_R > INSERTION_THRESHOLD) {
                if (len_R > OMP_TASK_THRESHOLD) {
                    #pragma omp task default(none) firstprivate(arr, p_right, high)
                    charlatana_v28_recursive(arr, p_right, high);
                } else {
                    charlatana_v28_recursive(arr, p_right, high);
                }
            } else {
                charlatana_insertion_sort(arr, p_right, high);
            }

            // Left is smaller: Loop back OR Sort and break
            if (len_L > INSERTION_THRESHOLD) {
                high = p_left; // Only loop back if it's large enough!
            } else {
                charlatana_insertion_sort(arr, low, p_left);
                break; // Left is sorted, right is handled. We are DONE.
            }
        } else {
            // Left is larger: Task, Recurse, or Sort
            if (len_L > INSERTION_THRESHOLD) {
                if (len_L > OMP_TASK_THRESHOLD) {
                    #pragma omp task default(none) firstprivate(arr, low, p_left)
                    charlatana_v28_recursive(arr, low, p_left);
                } else {
                    charlatana_v28_recursive(arr, low, p_left);
                }
            } else {
                charlatana_insertion_sort(arr, low, p_left);
            }

            // Right is smaller: Loop back OR Sort and break
            if (len_R > INSERTION_THRESHOLD) {
                low = p_right; // Only loop back if it's large enough!
            } else {
                charlatana_insertion_sort(arr, p_right, high);
```

```
                break; // Right is sorted, left is handled. We are DONE.
            }
        }
    }
}

/*
static inline void Charlatana_Sort(uint64_t* arr, size_t n) {
    if (n < 2) return;
    #pragma omp parallel
    {
        #pragma omp single
        {
            charlatana_V28_recursive(arr, 0, (int64_t)(n - 1));
        }
    }
}
*/

static inline void Charlatana_Sort(uint64_t* arr, size_t n) {
    //if (n < 2) return;

    if (n <= INSERTION_THRESHOLD) {
        charlatana_insertion_sort(arr, 0, (int64_t)(n - 1));
        return;
    }

    #pragma omp parallel
    {
        #pragma omp single
        {
            charlatana_v28_recursive(arr, 0, (int64_t)(n - 1));
        }
    }
}

#endif // CHARLATANA_V28_H

/*
Here is the visual breakdown of the branchless AVX2 partitioning pipeline.

Let's walk through an example where we load 4 elements, but only **Element A** and **Element C** are valid (e.g., they are smaller than the pivot), giving us a 4-bit mask of `0101` (decimal 5).

### Step 1: The Input & Comparison
We start with four 64-bit integers loaded into a single 256-bit AVX2 register. The CPU compares them against the pivot simultaneously.

```text
              256-bit AVX2 Register (Four 64-bit QWORDs)
+--------------+--------------+--------------+--------------+
|  QWORD 3 (D)  |  QWORD 2 (C)  |  QWORD 1 (B)  |  QWORD 0 (A)  |
+--------------+--------------+--------------+--------------+
       |              |              |              |
       v              v              v              v
    [ > Pivot ]    [ < Pivot ]    [ > Pivot ]    [ < Pivot ]
       |              |              |              |
       v              v              v              v
Mask:   0              1              0              1      --> Binary: 0101 (Dec: 5)
     (Discard)      (Keep)         (Discard)      (Keep)
```

Step 2: The "Squasher" (Permutation)
We use the mask `5` to look up Row 5 in the `QWORD_PACK_TABLE`.
This table feeds a permutation instruction that shuffles the 32-bit chunks. It grabs the valid 64-bit QWORDs (A and C) and packs them tightly to the right (the "left" side of the array logically, but right-side in little-endian register views).

```text
 QWORD_PACK_TABLE[5] = {0,1,  4,5,  2,3,  6,7}  <-- 32-bit chunk indices
                        |    |    |    |
      +-----------------+    |    |    +-----------------+
      |                      |    +---------+            |
      |      +---------------+              |            |
      |      |                              |            |
      v      v                              v            v
+--------------+--------------+--------------+--------------+
|  QWORD 3 (D)  |  QWORD 1 (B)  |  QWORD 2 (C)  |  QWORD 0 (A)  |
+--------------+--------------+--------------+--------------+
|====== GARBAGE (Ignored) ======|====== VALID (Squashed) =======|
```
```

Notice how Element C jumped over Element B so that A and C are now contiguous.

Step 3: The "Filter" (Masked Store)
Now we need to write the squashed register to main memory. We do a popcount on our `0101` mask, which equals ***2*** (two valid elements).

We look up Row 2 in the `MASK_STORE_TABLE`, which gives us `{-1LL, -1LL, 0, 0}`. The `maskstore` instruction uses this to act as a physical stencil over the register.

```text
              Squashed AVX2 Register
+---------------+---------------+---------------+---------------+
|    Garbage    |    Garbage    |  QWORD 2 (C)  |  QWORD 0 (A)  |
+---------------+---------------+---------------+---------------+
        |               |               |               |
        v               v               v               v
 MASK_STORE_TABLE[2] (The Stencil)
+---------------+---------------+---------------+---------------+
|       0       |       0       |     -1LL      |     -1LL      |
+---------------+---------------+---------------+---------------+
        |               |               |               |
   [ BLOCKED ]     [ BLOCKED ]   [ PASS THRU ]   [ PASS THRU ]
        |               |               |               |
        v               v               v               v
                Main Memory (Destination Array)
=============================================================
... | Unchanged |   Unchanged   |  Wrote C here |  Wrote A here | ...
=============================================================
                                          ^
                                          |
                                    Array Pointer
```

The Result
In just a few clock cycles, with ***zero branch instructions*** (no `if/else`), the CPU successfully identified the valid elements, compacted them, and cleanly wrote exactly two 64-bit chunks to memory. The array pointer is then advanced by 2, and the pipeline immediately processes the next 256 bits of data.
*/

/*
This is one of the most beautiful and brutal hacks in AVX2 programming.

What you are looking at is the manual, software-emulated version of AVX-512's `compressstore` instruction. Because AVX2 does ~~not~~ have a native way to take a sparse vector (where only some elements match your condition) and pack them tightly to the left side of the register before writing them to memory, we have to force the CPU to do it using these two lookup tables.

Here is exactly how this branchless pipeline works.

1. `QWORD_PACK_TABLE` (The "Squasher")
In the AVX2 version of Charlatana, we load ~~four 64-bit integers (QWORDs)~~ into a single 256-bit register. When we compare them to the pivot, we get a 4-bit mask (values from `0000` to `1111` in binary, or `0` to `15` in decimal).

This 4-bit mask tells us exactly which elements matched. For example, if the mask is `0101` (decimal 5), it means the 0th and 2nd elements matched, but the 1st and 3rd did not. We need to squash the 0th and 2nd elements to the front of the register.

To move data around in AVX2, we use `_mm256_permutevar8x32_epi32`. Notice the `32`? This instruction shuffles ~~32-bit chunks~~, not 64-bit chunks! A 256-bit register has eight 32-bit chunks. Since our 64-bit QWORDs are made of two 32-bit DWORDs each, we must move them in pairs:
~ ~~QWORD 0~~ is DWORDs `{0, 1}`
~ ~~QWORD 1~~ is DWORDs `{2, 3}`
~ ~~QWORD 2~~ is DWORDs `{4, 5}`
~ ~~QWORD 3~~ is DWORDs `{6, 7}`

The table has 16 rows (one for every possible 4-bit mask) and 8 columns (the instructions on where to put each 32-bit chunk).

~~Let's look at Row 5 (Mask `0101`):~~
```c
{0,1, 4,5, 2,3, 6,7}
```
~ The first two slots are `0,1`. This takes QWORD 0 and puts it in the 1st position.
~ The next two slots are `4,5`. This takes QWORD 2 and squashes it directly behind QWORD 0 into the 2nd position!
~ The rest (`2,3, 6,7`) is just garbage data pushed to the back. We don't care about it because it won't be written to memory anyway.

By looking up the row matching our 4-bit mask, we execute a single hardware shuffle that perfectly squashes all the valid data to the left side of the register in 1 clock cycle.

2. `MASK_STORE_TABLE` (The "Filter")
Now that all the valid data is packed cleanly to the left side of the 256-bit register, we need to write it to memory. But we ~only~ want to write the valid elements, not the garbage we pushed to the right.

To do this, we use `_mm256_maskstore_epi64`. This instruction looks at a control vector. If the most significant bit of a 64-bit lane in the control vector is `1` (which is represented in two's complement as `-1LL`, or `0xFFFFFFFFFFFFFFFF`), it writes that lane to RAM. If it's `0`, it safely ignores it.

The table has 5 rows because if you compare 4 elements, there are exactly 5 possible outcomes for the total number of matches (0, 1, 2, 3, or 4 matches). We find the row by doing a `__builtin_popcount(mask)` which simply counts how many `1`s are in our 4-bit mask.

~ ""Row 0 (0 matches):"" `{0, 0, 0, 0}` -> Writes nothing to memory.
~ ""Row 1 (1 match):"" `{-1LL, 0, 0, 0}` -> Writes exactly 1 QWORD to memory.
~ ""Row 2 (2 matches):"" `{-1LL, -1LL, 0, 0}` -> Writes exactly 2 QWORDs to memory.
~ ""Row 3 (3 matches):"" `{-1LL, -1LL, -1LL, 0}` -> Writes exactly 3 QWORDs to memory.
~ ""Row 4 (4 matches):"" `{-1LL, -1LL, -1LL, -1LL}` -> Writes all 4 QWORDs to memory.

The Summary of the Pipeline
When the CPU executes this code, the sequence is:
1. ""Compare:"" Yields a mask like `0101` (5).
2. ""Popcount:"" Counts two `1`s in the mask, so $k = 2$.
3. ""Shuffle:"" Loads Row 5 of `QWORD_PACK_TABLE`. Squashes the valid elements to the left.
4. ""Mask Load:"" Loads Row 2 of `MASK_STORE_TABLE` (because $k = 2$).
5. ""Store:"" Safely writes exactly the two valid 64-bit integers into the array.

This elegant combination of tables turns a branching, unpredictable `if-else` nightmare into a perfectly flat, uninterrupted SIMD mathematics pipeline. It is the secret to AVX2 sorting speeds!
*/

//]$ gcc -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28.asm benchmark28.c -S
// gcc version 15.2.1 20250808 (Red Hat 15.2.1-1) (GCC)
/*

*/

Charlatana_v29_AVX512_8bytes.h:
// Moving to AVX-512 is actually going to make the code much cleaner and potentially much faster.
// The beauty of AVX-512 is that Intel introduced native unsigned 64-bit compares and, most importantly, the Compress Store instruction (_mm512_mask_compressstoreu_epi64).
// In the AVX2 version (v28), you had to use massive lookup tables (QWORD_PACK_TABLE, MASK_STORE_TABLE) and messy cross-lane shuffles (_mm256_permutevar8x32_epi32) just to pack the elements to the left or right. AVX-512 does this natively in hardware. You hand it a mask of which elements are < pivot, and the CPU condenses and writes them sequentially to memory in one single instruction.
// Here is Charlatana_v29 (AVX-512). It uses the exact same algorithmic structure, thresholds, and buffer logic as your fastest v33 variant, but scales the processing block from 4 elements to 8 elements using pure AVX-512.

// Why this is structurally identical, but superior:
// - The 8-Element Jump: The while loop checks read_ptr_R - read_ptr_L >= 8 instead of 4. The buffers and conditions scale perfectly natively.
// - Native Comparisons: _mm512_cmp_epu64_mask calculates the boolean mask natively using the _MM_CMPINT_LT (Less Than) hardware flag, completely removing the awkward 0x8000000000000000ULL XOR hack from AVX2.
// - The Magic of Compress Store: _mm512_mask_compressstoreu_epi64 takes the 512-bit register and a mask, and dynamically squashes all the "true" elements sequentially into memory. This eliminates all permutation math, saving instructions and cycles.
// To benchmark this, be sure to compile with -mavx512f (or -march=native if running on a Zen 4 / Intel Sapphire Rapids or newer CPU).

```
#ifndef CHARLATANA_V29_H
#define CHARLATANA_V29_H

#include <immintrin.h>
#include <stdint.h>
#include <stddef.h>
#include <omp.h>

// DECOUPLED THRESHOLDS:
// Don't spawn OpenMP tasks for tiny chunks. It chokes the thread queue.
#define OMP_TASK_THRESHOLD 100000
#define INSERTION_THRESHOLD 64

// Notice: No permutation or mask tables required! AVX-512 handles this natively.

static void charlatana_insertion_sort(uint64_t* arr, int64_t low, int64_t high) {
    if (high <= low) return;

    uint64_t* base = arr + low;
    int64_t   n    = high - low + 1;

    int64_t min_idx = 0;
    for (int64_t k = 1; k < n; k++) {
        if (base[k] < base[min_idx]) min_idx = k;
    }
    uint64_t tmp = base[0];
    base[0]      = base[min_idx];
    base[min_idx]= tmp;

    for (int64_t i = 1; i < n; i++) {
        uint64_t  key  = base[i];
        uint64_t* jptr = base + i - 1;

        while (*jptr > key) {
            *(jptr + 1) = *jptr;
            jptr--;
        }
        *(jptr + 1) = key;
    }
}

static void charlatana_v29_recursive(uint64_t* arr, int64_t low, int64_t high) {
    while (low < high) {

        int64_t delta = high - low;
        uint64_t v1   = arr[low   + delta / 4];
        uint64_t v2   = arr[low   + delta / 2];
        uint64_t v3   = arr[high  - delta / 4];

        // Fast Branchless Median-of-3 using XOR tricks
        uint64_t mask_min_xy = -(uint64_t)(v1 > v2);
        uint64_t mask_max_xy = -(uint64_t)(v1 < v2);
        uint64_t min_xy = v1 ^ ((v1 ^ v2) & mask_min_xy);
        uint64_t max_xy = v1 ^ ((v1 ^ v2) & mask_max_xy);

        uint64_t mask_min = -(uint64_t)(min_xy > v3);
        uint64_t overall_min = min_xy ^ ((min_xy ^ v3) & mask_min);

        uint64_t mask_max = -(uint64_t)(max_xy < v3);
        uint64_t overall_max = max_xy ^ ((max_xy ^ v3) & mask_max);
```

```
    uint64_t pivot = v1 ^ v2 ^ v3 ^ overall_min ^ overall_max;

    // Broadcast pivot to all 8 lanes of the 512-bit register
    __m512i v_pivot = _mm512_set1_epi64(pivot);

    uint64_t* read_ptr_L = arr + low;
    uint64_t* write_ptr_L = arr + low;
    uint64_t* read_ptr_R = arr + high + 1;
    uint64_t* write_ptr_R = arr + high + 1;

    uint64_t buf_L[256];
    uint64_t buf_R[256];
    int count_L = 0;
    int count_R = 0;

    // Process 8 elements (512 bits) per iteration instead of 4
    while (read_ptr_R - read_ptr_L >= 8) {
        if (count_L > count_R) {
            // Read 8 elements from Left
            __m512i v = _mm512_loadu_si512(read_ptr_L);
            read_ptr_L += 8;

            // Native AVX-512 Unsigned Compare (Less Than)
            __mmask8 mask_lt = _mm512_cmp_epu64_mask(v, v_pivot, _MM_CMPINT_LT);
            int k_lt = __builtin_popcount(mask_lt);
            // Hardware Compress Store: Packs matched elements into contiguous memory
            _mm512_mask_compressstoreu_epi64(write_ptr_L, mask_lt, v);
            write_ptr_L += k_lt;

            // Native AVX-512 Unsigned Compare (Greater Than)
            __mmask8 mask_gt = _mm512_cmp_epu64_mask(v, v_pivot, _MM_CMPINT_GT);
            int k_gt = __builtin_popcount(mask_gt);
            _mm512_mask_compressstoreu_epi64(buf_R + count_R, mask_gt, v);
            count_R += k_gt;

            int space_L = read_ptr_L - write_ptr_L;
            if (count_L > 0 && space_L >= count_L) {
                for (int k = 0; k < count_L; k++) write_ptr_L[k] = buf_L[k];
                write_ptr_L += count_L;
                count_L = 0;
            }
        } else {
            // Read 8 elements from Right
            read_ptr_R -= 8;
            __m512i v = _mm512_loadu_si512(read_ptr_R);

            // Native AVX-512 Unsigned Compare (Greater Than)
            __mmask8 mask_gt = _mm512_cmp_epu64_mask(v, v_pivot, _MM_CMPINT_GT);
            int k_gt = __builtin_popcount(mask_gt);
            write_ptr_R -= k_gt;
            _mm512_mask_compressstoreu_epi64(write_ptr_R, mask_gt, v);

            // Native AVX-512 Unsigned Compare (Less Than)
            __mmask8 mask_lt = _mm512_cmp_epu64_mask(v, v_pivot, _MM_CMPINT_LT);
            int k_lt = __builtin_popcount(mask_lt);
            _mm512_mask_compressstoreu_epi64(buf_L + count_L, mask_lt, v);
            count_L += k_lt;

            int space_R = write_ptr_R - read_ptr_R;
            if (count_R > 0 && space_R >= count_R) {
                write_ptr_R -= count_R;
                for (int k = 0; k < count_R; k++) write_ptr_R[k] = buf_R[k];
                count_R = 0;
            }
        }
    }

    // Scalar cleanup for the remaining < 8 elements
    while (read_ptr_L < read_ptr_R) {
        uint64_t val = *read_ptr_L;
        int is_lt = (val < pivot);
        int is_gt = (val > pivot);

        *write_ptr_L = val;
        write_ptr_L += is_lt;

        buf_R[count_R] = val;
        count_R += is_gt;
```

```
            read_ptr_L++;
        }

        for (int k = 0; k < count_L; k++) *write_ptr_L++ = buf_L[k];
        write_ptr_R -= count_R;
        for (int k = 0; k < count_R; k++) write_ptr_R[k] = buf_R[k];

        uint64_t* end_lt = write_ptr_L;
        uint64_t* start_gt = write_ptr_R;

        // Elements strictly equal to the pivot
        while (write_ptr_L < write_ptr_R) {
            *write_ptr_L++ = pivot;
        }

        // --- Streamlined Tail Routing (identical to fastest v33 logic) ---
        int64_t p_left = (end_lt - arr) - 1;
        int64_t p_right = (start_gt - arr);

        int64_t len_L = p_left - low + 1;
        int64_t len_R = high - p_right + 1;

        if (len_L < len_R) {
            if (len_R > INSERTION_THRESHOLD) {
                if (len_R > OMP_TASK_THRESHOLD) {
                    #pragma omp task default(none) firstprivate(arr, p_right, high)
                    charlatana_v29_recursive(arr, p_right, high);
                } else {
                    charlatana_v29_recursive(arr, p_right, high);
                }
            } else {
                charlatana_insertion_sort(arr, p_right, high);
            }

            if (len_L > INSERTION_THRESHOLD) {
                high = p_left;
            } else {
                charlatana_insertion_sort(arr, low, p_left);
                break;
            }
        } else {
            if (len_L > INSERTION_THRESHOLD) {
                if (len_L > OMP_TASK_THRESHOLD) {
                    #pragma omp task default(none) firstprivate(arr, low, p_left)
                    charlatana_v29_recursive(arr, low, p_left);
                } else {
                    charlatana_v29_recursive(arr, low, p_left);
                }
            } else {
                charlatana_insertion_sort(arr, low, p_left);
            }

            if (len_R > INSERTION_THRESHOLD) {
                low = p_right;
            } else {
                charlatana_insertion_sort(arr, p_right, high);
                break;
            }
        }
    }
}

static inline void Charlatana_Sort(uint64_t* arr, size_t n) {
    if (n <= INSERTION_THRESHOLD) {
        charlatana_insertion_sort(arr, 0, (int64_t)(n - 1));
        return;
    }

    #pragma omp parallel
    {
        #pragma omp single
        {
            charlatana_v29_recursive(arr, 0, (int64_t)(n - 1));
        }
    }
}

#endif // CHARLATANA_V29_H
```

benchmark28.c:

```
/*
COMPILE WITH OPENMP:
$ gcc -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c

$ x86_64-w64-mingw32-gcc -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28.exe benchmark28.c
$ cp /usr/x86_64-w64-mingw32/sys-root/mingw/bin/libgomp-1.dll .
$ cp /usr/x86_64-w64-mingw32/sys-root/mingw/bin/libwinpthread-1.dll .
$ cp /usr/x86_64-w64-mingw32/sys-root/mingw/bin/libgcc_s_seh-1.dll .

Usage:
  ./benchmark <file_to_sort> <algorithm_choice>

Algorithms:
  1 = Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
  2 = Standard C qsort()
*/

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <time.h>
#include <stdbool.h>
#include <omp.h>
#include <locale.h>          // NEW: for comma-formatted thousands

#include "Charlatana_v28.h"

int compare_uint64(const void* a, const void* b) {
    uint64_t val_a = *(const uint64_t*)a;
    uint64_t val_b = *(const uint64_t*)b;
    if (val_a < val_b) return -1;
    if (val_a > val_b) return 1;
    return 0;
}

bool is_sorted(uint64_t* arr, size_t n) {
    for (size_t i = 1; i < n; i++) {
        if (arr[i - 1] > arr[i]) return false;
    }
    return true;
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <file_to_sort> <algorithm>\n", argv[0]);
        fprintf(stderr, "Algorithms:\n  1 = Charlatana_v28\n  2 = Standard qsort()\n");
        return EXIT_FAILURE;
    }

    const char* filename = argv[1];
    int algo_choice = atoi(argv[2]);

    FILE *file = fopen(filename, "rb");
    if (!file) {
        perror("Error opening file");
        return EXIT_FAILURE;
    }

    fseek(file, 0, SEEK_END);
    long filesize = ftell(file);
    fseek(file, 0, SEEK_SET);

    if (filesize < 8) {
        fprintf(stderr, "Error: File is too small.\n");
        fclose(file);
        return EXIT_FAILURE;
    }

    uint8_t *file_buffer = (uint8_t*)malloc(filesize);
    fread(file_buffer, 1, filesize, file);
    fclose(file);

    size_t num_keys = filesize - 8 + 1;
    printf("Extracting %zu keys...\n", num_keys);

    uint64_t *keys = (uint64_t*)malloc(num_keys * sizeof(uint64_t));
    for (size_t i = 0; i < num_keys; i++) {
```

```
        memcpy(&keys[i], &file_buffer[i], sizeof(uint64_t));
    }
    free(file_buffer);

    /* === NEW: enable comma formatting for performance numbers === */
    setlocale(LC_NUMERIC, "");

    printf("\n--- Starting Benchmark ---\n");
    double start_time = omp_get_wtime();

    if (algo_choice == 1) {
        int threads = omp_get_max_threads();
        printf("Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)\n");
        printf("OpenMP Engine: Utilizing up to %d Hardware Threads\n", threads);
        Charlatana_Sort(keys, num_keys);
    }
    else if (algo_choice == 2) {
        printf("Algorithm: Standard C qsort() [Single Threaded]\n");
        qsort(keys, num_keys, sizeof(uint64_t), compare_uint64);
    }

    double end_time = omp_get_wtime();
    double elapsed = end_time - start_time;

    printf("Sort Time: %.4f seconds\n", elapsed);

    /* === NEW: Total Performance report (Total_keys / OpenMPtime) === */
    if (elapsed > 0.0) {
        uint64_t keys_per_sec = (uint64_t)(num_keys / elapsed);
        printf("Total Performance: %'llu keys-per-second\n", (unsigned long long)keys_per_sec);
    } else {
        printf("Total Performance: N/A keys-per-second\n");
    }

    /* === UNIQUE KEYS COUNT (after sorting) === */
    size_t unique_keys = 0;
    if (num_keys > 0) {
        unique_keys = 1;
        for (size_t i = 1; i < num_keys; i++) {
            if (keys[i] != keys[i - 1]) {
                unique_keys++;
            }
        }
    }
    printf("Unique keys: %'zu\n", unique_keys);

    printf("\n--- Verification ---\n");
    bool is_properly_sorted = is_sorted(keys, num_keys);
    printf("Array properly sorted: %s\n", is_properly_sorted ? "YES" : "NO");

    free(keys);
    return EXIT_SUCCESS;
}

// Core i7-11850H (Tiger Lake-H). 128GB DDR4 3200MHz

/*
[sanmayce@djudjeto Quicksort_benchmark_2024-Nov-28_update]$ cat bench.sh
echo Charlatana_v28:
gcc -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 SUPRAPIG_SPETSNAZ-in-Russian_\(FB2-1012\).tar 1
clang -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 SUPRAPIG_SPETSNAZ-in-Russian_\(FB2-1012\).tar 1

/bin/time -v ./benchmark28 SUPRAPIG_SPETSNAZ-in-Russian_\(FB2-1012\).tar 1

gcc -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 SUPRAPIG_github.com_Sefaria-Project_2024-Jun-01_16833-TXTs.tar 1
clang -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 SUPRAPIG_github.com_Sefaria-Project_2024-Jun-01_16833-TXTs.tar 1

/bin/time -v ./benchmark28 SUPRAPIG_github.com_Sefaria-Project_2024-Jun-01_16833-TXTs.tar 1

gcc -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 Nobara-43-GNOME-2026-01-26.iso 1
clang -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 Nobara-43-GNOME-2026-01-26.iso 1

/bin/time -v ./benchmark28 Nobara-43-GNOME-2026-01-26.iso 1
```

```
gcc -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 SUPRAPIG_news.ycombinator.com-items-201909_21M-news.ycombinator.com.jsonl 1
clang -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 SUPRAPIG_news.ycombinator.com-items-201909_21M-news.ycombinator.com.jsonl 1

/bin/time -v ./benchmark28 SUPRAPIG_news.ycombinator.com-items-201909_21M-news.ycombinator.com.jsonl 1

gcc -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 Brawl_in_cell_block_99_\(2017\)_UHD_BDRip_720p.mkv 1
clang -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 Brawl_in_cell_block_99_\(2017\)_UHD_BDRip_720p.mkv 1

/bin/time -v ./benchmark28 Brawl_in_cell_block_99_\(2017\)_UHD_BDRip_720p.mkv 1

echo
echo qsort:
gcc -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 SUPRAPIG_SPETSNAZ-in-Russian_\(FB2-1012\).tar 2
clang -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 SUPRAPIG_SPETSNAZ-in-Russian_\(FB2-1012\).tar 2

gcc -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 SUPRAPIG_github.com_Sefaria-Project_2024-Jun-01_16833-TXTs.tar 2
clang -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 SUPRAPIG_github.com_Sefaria-Project_2024-Jun-01_16833-TXTs.tar 2

gcc -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 Nobara-43-GNOME-2026-01-26.iso 2
clang -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 Nobara-43-GNOME-2026-01-26.iso 2

gcc -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 SUPRAPIG_news.ycombinator.com-items-201909_21M-news.ycombinator.com.jsonl 2
clang -O3 -mavx2 -fopenmp -std=gnu11 -o benchmark28 benchmark28.c
perf stat -d ./benchmark28 SUPRAPIG_news.ycombinator.com-items-201909_21M-news.ycombinator.com.jsonl 2

[sanmayce@djudjeto Quicksort_benchmark_2024-Nov-28_update]$
*/

/*
[sanmayce@djudjeto Quicksort_benchmark_2024-Nov-28_update]$ sh bench.sh
Charlatana_v28:
Extracting 998340601 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 10.3061 seconds
Total Performance: 96,868,533 keys-per-second
Unique keys: 209,106,265

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 SUPRAPIG_SPETSNAZ-in-Russian_(FB2-1012).tar 1':

        122,470.51 msec task-clock:u                     #    9.010 CPUs utilized
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
         1,950,460      page-faults:u                    #   15.926 K/sec
   344,165,215,378      instructions:u                   #    1.03  insn per cycle              (90.91%)
   332,538,057,800      cycles:u                         #    2.715 GHz                         (90.91%)
    44,137,185,917      branches:u                       #  360.390 M/sec                       (90.91%)
     2,501,861,517      branch-misses:u                  #    5.67% of all branches             (81.84%)
                        TopdownL1                #     4.9 %  tma_backend_bound
                                                 #    91.0 %  tma_bad_speculation
                                                 #    -0.0 %  tma_frontend_bound
                                                 #     4.2 %  tma_retiring             (90.92%)
    53,733,152,687      L1-dcache-loads:u                #  438.744 M/sec                       (90.90%)
     3,183,538,531      L1-dcache-load-misses:u          #    5.92% of all L1-dcache accesses   (90.91%)
        61,238,102      LLC-loads:u                      #  500.023 K/sec                       (90.90%)
        58,056,977      LLC-load-misses:u                #   94.81% of all LL-cache accesses    (81.83%)

      13.592276302 seconds time elapsed

     115.820343000 seconds user
       1.995976000 seconds sys
```

```
Extracting 998340601 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 11.9045 seconds
Total Performance: 83,862,583 keys-per-second
Unique keys: 209,106,265

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 SUPRAPIG_SPETSNAZ-in-Russian_(FB2-1012).tar 1':

       178,036.30 msec task-clock:u                     #   11.811 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
        1,950,704      page-faults:u                    #   10.957 K/sec
  457,680,462,264      instructions:u                   #    0.95  insn per cycle              (90.91%)
  481,166,252,393      cycles:u                         #    2.703 GHz                         (90.90%)
   72,842,294,078      branches:u                       #  409.143 M/sec                       (90.90%)
    3,278,125,285      branch-misses:u                  #    4.50% of all branches             (81.82%)
                       TopdownL1                 #      9.6 %  tma_backend_bound
                                                 #     83.8 %  tma_bad_speculation
                                                 #      0.6 %  tma_frontend_bound
                                                 #      6.0 %  tma_retiring             (90.91%)
   82,113,273,218      L1-dcache-loads:u                #  461.216 M/sec                       (90.91%)
    3,970,894,539      L1-dcache-load-misses:u          #    4.84% of all L1-dcache accesses   (90.91%)
      482,400,901      LLC-loads:u                      #    2.710 M/sec                       (90.91%)
       61,336,250      LLC-load-misses:u                #   12.71% of all LL-cache accesses    (81.82%)

     15.074071314 seconds time elapsed

    169.396583000 seconds user
      2.583716000 seconds sys


Extracting 998340601 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 11.3617 seconds
Total Performance: 87,868,644 keys-per-second
Unique keys: 209,106,265

--- Verification ---
Array properly sorted: YES
        Command being timed: "./benchmark28 SUPRAPIG_SPETSNAZ-in-Russian_(FB2-1012).tar 1"
        User time (seconds): 165.71
        System time (seconds): 2.51
        Percent of CPU this job got: 1164%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:14.44
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 8776320
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 2194453
        Voluntary context switches: 81
        Involuntary context switches: 2779
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
Extracting 5694801913 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 61.0153 seconds
Total Performance: 93,334,006 keys-per-second
```

```
Unique keys: 82,181,812

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 SUPRAPIG_github.com_Sefaria-Project_2024-Jun-01_16833-TXTs.tar 1':

       751,688.26 msec task-clock:u                     #    9.378 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
       11,123,421      page-faults:u                    #   14.798 K/sec
1,743,432,645,885      instructions:u                    #    0.80  insn per cycle              (90.91%)
2,180,343,479,303      cycles:u                         #    2.901 GHz                         (90.91%)
  220,792,301,521      branches:u                       #  293.729 M/sec                       (90.91%)
   11,859,068,243      branch-misses:u                  #    5.37% of all branches             (81.82%)
                       TopdownL1                #      3.1 %  tma_backend_bound
                                                #     94.9 %  tma_bad_speculation
                                                #     -0.2 %  tma_frontend_bound
                                                #      2.1 %  tma_retiring             (90.91%)
  271,728,016,219      L1-dcache-loads:u                #  361.490 M/sec                       (90.91%)
   19,249,010,118      L1-dcache-load-misses:u          #    7.08% of all L1-dcache accesses   (90.91%)
      371,245,722      LLC-loads:u                      #  493.883 K/sec                       (90.91%)
      356,128,215      LLC-load-misses:u                #   95.93% of all LL-cache accesses    (81.82%)

     80.158144755 seconds time elapsed

    711.969378000 seconds user
     11.245417000 seconds sys


Extracting 5694801913 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 67.8278 seconds
Total Performance: 83,959,736 keys-per-second
Unique keys: 82,181,812

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 SUPRAPIG_github.com_Sefaria-Project_2024-Jun-01_16833-TXTs.tar 1':

       965,591.91 msec task-clock:u                     #   11.214 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
       11,123,475      page-faults:u                    #   11.520 K/sec
2,222,200,273,818      instructions:u                    #    0.78  insn per cycle              (90.91%)
2,844,712,749,495      cycles:u                         #    2.946 GHz                         (90.91%)
  342,475,044,733      branches:u                       #  354.679 M/sec                       (90.91%)
   16,112,050,045      branch-misses:u                  #    4.70% of all branches             (81.82%)
                       TopdownL1                #      9.1 %  tma_backend_bound
                                                #     89.2 %  tma_bad_speculation
                                                #     -0.1 %  tma_frontend_bound
                                                #      1.9 %  tma_retiring             (90.91%)
  380,869,945,471      L1-dcache-loads:u                #  394.442 M/sec                       (90.91%)
   22,799,286,293      L1-dcache-load-misses:u          #    5.99% of all L1-dcache accesses   (90.91%)
    2,332,229,875      LLC-loads:u                      #    2.415 M/sec                       (90.91%)
      411,882,245      LLC-load-misses:u                #   17.66% of all LL-cache accesses    (81.82%)

     86.106339017 seconds time elapsed

    920.421033000 seconds user
     13.862213000 seconds sys


Extracting 5694801913 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 65.3394 seconds
Total Performance: 87,157,184 keys-per-second
Unique keys: 82,181,812

--- Verification ---
Array properly sorted: YES
        Command being timed: "./benchmark28 SUPRAPIG_github.com_Sefaria-Project_2024-Jun-01_16833-TXTs.tar 1"
```

```
        User time (seconds): 905.38
        System time (seconds): 13.23
        Percent of CPU this job got: 1100%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 1:23.45
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 50053784
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 12513824
        Voluntary context switches: 90
        Involuntary context switches: 21991
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
Extracting 6126911481 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 89.9750 seconds
Total Performance: 68,095,738 keys-per-second
Unique keys: 5,726,784,672

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 Nobara-43-GNOME-2026-01-26.iso 1':

     1,161,700.34 msec task-clock:u                     #   10.535 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
           11,967,495      page-faults:u                #   10.302 K/sec
 3,411,691,180,821      instructions:u                   #    1.19  insn per cycle              (90.91%)
 2,865,372,137,569      cycles:u                         #    2.467 GHz                         (90.91%)
   445,501,001,379      branches:u                       #  383.490 M/sec                       (90.91%)
    28,925,507,830      branch-misses:u                  #    6.49% of all branches             (81.82%)
                        TopdownL1                #      2.6 %  tma_backend_bound
                                                 #     96.3 %  tma_bad_speculation
                                                 #     -0.4 %  tma_frontend_bound
                                                 #      1.5 %  tma_retiring             (90.91%)
   536,515,293,791      L1-dcache-loads:u                #  461.836 M/sec                       (90.91%)
    22,678,327,761      L1-dcache-load-misses:u          #    4.23% of all L1-dcache accesses   (90.91%)
       408,214,933      LLC-loads:u                      #  351.394 K/sec                       (90.91%)
       384,924,870      LLC-load-misses:u                #   94.29% of all LL-cache accesses    (81.82%)

     110.265365940 seconds time elapsed

    1102.491218000 seconds user
      12.428618000 seconds sys


Extracting 6126911481 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 95.6674 seconds
Total Performance: 64,043,895 keys-per-second
Unique keys: 5,726,784,672

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 Nobara-43-GNOME-2026-01-26.iso 1':

     1,413,945.88 msec task-clock:u                     #   12.319 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
           11,967,502      page-faults:u                #    8.464 K/sec
 3,904,650,242,139      instructions:u                   #    1.07  insn per cycle              (90.91%)
 3,643,146,084,414      cycles:u                         #    2.577 GHz                         (90.91%)
```

```
   602,946,679,679      branches:u                      #  426.428 M/sec                    (90.91%)
    36,568,971,171      branch-misses:u                 #    6.07% of all branches          (81.82%)
                        TopdownL1                 #      5.4 %  tma_backend_bound
                                                  #     93.1 %  tma_bad_speculation
                                                  #     -0.3 %  tma_frontend_bound
                                                  #      1.8 %  tma_retiring             (90.91%)
   609,845,724,000      L1-dcache-loads:u               #  431.308 M/sec                    (90.91%)
    26,796,900,097      L1-dcache-load-misses:u         #    4.39% of all L1-dcache accesses  (90.91%)
     2,651,730,973      LLC-loads:u                     #    1.875 M/sec                    (90.91%)
       479,740,826      LLC-load-misses:u               #   18.09% of all LL-cache accesses   (81.82%)

     114.780699032 seconds time elapsed

    1348.881731000 seconds user
      14.567139000 seconds sys


Extracting 6126911481 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 92.7155 seconds
Total Performance: 66,082,946 keys-per-second
Unique keys: 5,726,784,672

--- Verification ---
Array properly sorted: YES
        Command being timed: "./benchmark28 Nobara-43-GNOME-2026-01-26.iso 1"
        User time (seconds): 1344.50
        System time (seconds): 13.91
        Percent of CPU this job got: 1220%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 1:51.28
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 53851636
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 13463343
        Voluntary context switches: 85
        Involuntary context switches: 22451
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
Extracting 9758687449 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 110.5515 seconds
Total Performance: 88,272,808 keys-per-second
Unique keys: 316,438,651

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 SUPRAPIG_news.ycombinator.com-items-201909_21M-news.ycombinator.com.jsonl 1':

     1,312,222.97 msec task-clock:u                     #    9.187 CPUs utilized
                0      context-switches:u              #    0.000 /sec
                0      cpu-migrations:u                #    0.000 /sec
       19,060,747      page-faults:u                   #   14.526 K/sec
 3,281,791,683,039      instructions:u                  #    0.88  insn per cycle             (90.91%)
 3,745,673,554,108      cycles:u                        #    2.854 GHz                        (90.91%)
   415,754,313,177      branches:u                      #  316.832 M/sec                      (90.91%)
    23,635,494,539      branch-misses:u                 #    5.68% of all branches          (81.82%)
                        TopdownL1                 #      2.1 %  tma_backend_bound
                                                  #     95.8 %  tma_bad_speculation
                                                  #     -0.2 %  tma_frontend_bound
                                                  #      2.3 %  tma_retiring             (90.91%)
   510,774,423,374      L1-dcache-loads:u               #  389.244 M/sec                    (90.91%)
    33,570,469,761      L1-dcache-load-misses:u         #    6.57% of all L1-dcache accesses  (90.91%)
```

```
       628,060,941      LLC-loads:u                   #  478.624 K/sec                    (90.91%)
       600,159,577      LLC-load-misses:u             #   95.56% of all LL-cache accesses    (81.82%)

     142.827376306 seconds time elapsed

    1242.486656000 seconds user
      19.575721000 seconds sys


Extracting 9758687449 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 122.1156 seconds
Total Performance: 79,913,516 keys-per-second
Unique keys: 316,438,651

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 SUPRAPIG_news.ycombinator.com-items-201909_21M-news.ycombinator.com.jsonl 1':

      1,741,460.14 msec task-clock:u                     #   11.380 CPUs utilized
                 0      context-switches:u             #    0.000 /sec
                 0      cpu-migrations:u               #    0.000 /sec
        19,060,768      page-faults:u                  #   10.945 K/sec
 4,127,886,241,813      instructions:u                 #    0.82  insn per cycle              (90.91%)
 5,013,116,694,929      cycles:u                       #    2.879 GHz                         (90.91%)
   633,697,859,148      branches:u                     #  363.889 M/sec                       (90.91%)
    31,703,244,840      branch-misses:u                #    5.00% of all branches             (81.82%)
                        TopdownL1                 #      6.5 %  tma_backend_bound
                                                  #     91.3 %  tma_bad_speculation
                                                  #     -0.2 %  tma_frontend_bound
                                                  #      2.4 %  tma_retiring             (90.91%)
   697,655,133,967      L1-dcache-loads:u              #  400.615 M/sec                       (90.91%)
    40,024,348,259      L1-dcache-load-misses:u        #    5.74% of all L1-dcache accesses   (90.91%)
     4,283,577,242      LLC-loads:u                    #    2.460 M/sec                       (90.91%)
       785,704,061      LLC-load-misses:u              #   18.34% of all LL-cache accesses    (81.82%)

     153.022548463 seconds time elapsed

    1660.024199000 seconds user
      23.463559000 seconds sys


Extracting 9758687449 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 119.6554 seconds
Total Performance: 81,556,594 keys-per-second
Unique keys: 316,438,651

--- Verification ---
Array properly sorted: YES
        Command being timed: "./benchmark28 SUPRAPIG_news.ycombinator.com-items-201909_21M-news.ycombinator.com.jsonl 1"
        User time (seconds): 1666.72
        System time (seconds): 22.68
        Percent of CPU this job got: 1128%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 2:29.67
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 85771508
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 21443271
        Voluntary context switches: 84
        Involuntary context switches: 32954
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
```

```
        Exit status: 0
Extracting 11402523458 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 177.7942 seconds
Total Performance: 64,133,278 keys-per-second
Unique keys: 11,116,109,642

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 Brawl_in_cell_block_99_(2017)_UHD_BDRip_720p.mkv 1':

     2,244,660.03 msec task-clock:u                     #   10.464 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
       22,271,464      page-faults:u                    #    9.922 K/sec
 6,480,248,934,235      instructions:u                   #    1.17  insn per cycle              (90.91%)
 5,530,252,990,842      cycles:u                         #    2.464 GHz                         (90.91%)
   840,540,639,954      branches:u                       #  374.462 M/sec                       (90.91%)
    54,769,045,916      branch-misses:u                  #    6.52% of all branches             (81.82%)
                        TopdownL1                 #      2.5 %  tma_backend_bound
                                                  #     96.3 %  tma_bad_speculation
                                                  #     -0.4 %  tma_frontend_bound
                                                  #      1.6 %  tma_retiring             (90.91%)
 1,016,166,795,867      L1-dcache-loads:u                #  452.704 M/sec                       (90.91%)
    43,974,072,136      L1-dcache-load-misses:u          #    4.33% of all L1-dcache accesses   (90.91%)
       777,620,659      LLC-loads:u                      #  346.431 K/sec                       (90.91%)
       733,211,209      LLC-load-misses:u                #   94.29% of all LL-cache accesses    (81.82%)

     214.510679143 seconds time elapsed

    2129.802199000 seconds user
      22.321096000 seconds sys


Extracting 11402523458 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 183.6663 seconds
Total Performance: 62,082,826 keys-per-second
Unique keys: 11,116,109,642

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 Brawl_in_cell_block_99_(2017)_UHD_BDRip_720p.mkv 1':

     2,754,449.11 msec task-clock:u                     #   12.489 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
       22,271,448      page-faults:u                    #    8.086 K/sec
 7,573,332,742,688      instructions:u                   #    1.05  insn per cycle              (90.91%)
 7,230,074,546,644      cycles:u                         #    2.625 GHz                         (90.91%)
 1,170,435,576,183      branches:u                       #  424.925 M/sec                       (90.91%)
    68,887,829,627      branch-misses:u                  #    5.89% of all branches             (81.82%)
                        TopdownL1                 #      4.7 %  tma_backend_bound
                                                  #     95.0 %  tma_bad_speculation
                                                  #     -0.5 %  tma_frontend_bound
                                                  #      0.8 %  tma_retiring             (90.91%)
 1,206,217,487,617      L1-dcache-loads:u                #  437.916 M/sec                       (90.91%)
    53,180,816,844      L1-dcache-load-misses:u          #    4.41% of all L1-dcache accesses   (90.91%)
     5,810,570,550      LLC-loads:u                      #    2.110 M/sec                       (90.91%)
       923,528,733      LLC-load-misses:u                #   15.89% of all LL-cache accesses    (81.82%)

     220.544431233 seconds time elapsed

    2630.378806000 seconds user
      28.520519000 seconds sys


Extracting 11402523458 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
```

```
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 184.2720 seconds
Total Performance: 61,878,751 keys-per-second
Unique keys: 11,116,109,642

--- Verification ---
Array properly sorted: YES
        Command being timed: "./benchmark28 Brawl_in_cell_block_99_(2017)_UHD_BDRip_720p.mkv 1"
        User time (seconds): 2711.55
        System time (seconds): 27.43
        Percent of CPU this job got: 1247%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 3:39.53
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 100219504
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 25055281
        Voluntary context switches: 456
        Involuntary context switches: 37648
        Swaps: 0
        File system inputs: 3414624
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
*/

/*
qsort:
Extracting 998340601 keys...

--- Starting Benchmark ---
Algorithm: Standard C qsort() [Single Threaded]
Sort Time: 105.6141 seconds
Total Performance: 9,452,719 keys-per-second

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 SUPRAPIG_SPETSNAZ-in-Russian_(FB2-1012).tar 2':

        108,437.18 msec task-clock:u                     #    1.000 CPUs utilized
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
         3,899,848      page-faults:u                    #   35.964 K/sec
   762,455,590,544      instructions:u                   #    1.53  insn per cycle              (90.91%)
   499,221,186,903      cycles:u                         #    4.604 GHz                         (90.91%)
   162,943,125,354      branches:u                       #    1.503 G/sec                       (90.91%)
    10,169,192,584      branch-misses:u                  #    6.24% of all branches             (81.82%)
                        TopdownL1                 #      2.3 %  tma_backend_bound
                                                  #     97.5 %  tma_bad_speculation
                                                  #     -0.4 %  tma_frontend_bound
                                                  #      0.6 %  tma_retiring             (90.91%)
   205,306,023,257      L1-dcache-loads:u                #    1.893 G/sec                       (90.91%)
     8,233,276,053      L1-dcache-load-misses:u          #    4.01% of all L1-dcache accesses   (90.91%)
       123,772,717      LLC-loads:u                      #    1.141 M/sec                       (90.91%)
        70,745,842      LLC-load-misses:u                #   57.16% of all LL-cache accesses    (81.82%)

     108.464918100 seconds time elapsed

     104.177880000 seconds user
       3.499134000 seconds sys

Extracting 998340601 keys...

--- Starting Benchmark ---
Algorithm: Standard C qsort() [Single Threaded]
Sort Time: 112.6477 seconds
Total Performance: 8,862,506 keys-per-second

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 SUPRAPIG_SPETSNAZ-in-Russian_(FB2-1012).tar 2':
```

```
        115,375.90 msec task-clock:u                     #    1.000 CPUs utilized
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
         3,899,860      page-faults:u                    #   33.801 K/sec
   701,063,063,000      instructions:u                   #    1.32  insn per cycle              (90.91%)
   532,424,231,820      cycles:u                         #    4.615 GHz                         (90.91%)
   133,608,491,733      branches:u                       #    1.158 G/sec                       (90.91%)
    10,521,957,591      branch-misses:u                  #    7.88% of all branches             (81.82%)
                        TopdownL1                 #      2.2 %  tma_backend_bound
                                                  #     97.7 %  tma_bad_speculation
                                                  #     -0.3 %  tma_frontend_bound
                                                  #      0.4 %  tma_retiring             (90.91%)
   205,304,135,347      L1-dcache-loads:u                #    1.779 G/sec                       (90.91%)
     8,230,341,680      L1-dcache-load-misses:u          #    4.01% of all L1-dcache accesses   (90.91%)
       122,294,292      LLC-loads:u                      #    1.060 M/sec                       (90.91%)
        69,841,008      LLC-load-misses:u                #   57.11% of all LL-cache accesses    (81.82%)

     115.404180468 seconds time elapsed

     111.157058000 seconds user
       3.433995000 seconds sys

Extracting 5694801913 keys...

--- Starting Benchmark ---
Algorithm: Standard C qsort() [Single Threaded]
Sort Time: 610.9921 seconds
Total Performance: 9,320,582 keys-per-second

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 SUPRAPIG_github.com_Sefaria-Project_2024-Jun-01_16833-TXTs.tar 2':

        627,104.34 msec task-clock:u                     #    1.000 CPUs utilized
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
        22,245,402      page-faults:u                    #   35.473 K/sec
 4,753,616,681,616      instructions:u                   #    1.71  insn per cycle              (90.91%)
 2,776,162,721,327      cycles:u                         #    4.427 GHz                         (90.91%)
 1,005,835,596,082      branches:u                       #    1.604 G/sec                       (90.91%)
    49,502,790,940      branch-misses:u                  #    4.92% of all branches             (81.82%)
                        TopdownL1                 #      2.0 %  tma_backend_bound
                                                  #     98.2 %  tma_bad_speculation
                                                  #     -0.3 %  tma_frontend_bound
                                                  #      0.2 %  tma_retiring             (90.91%)
 1,268,440,096,307      L1-dcache-loads:u                #    2.023 G/sec                       (90.91%)
    52,490,702,203      L1-dcache-load-misses:u          #    4.14% of all L1-dcache accesses   (90.91%)
       806,113,458      LLC-loads:u                      #    1.285 M/sec                       (90.91%)
       486,524,415      LLC-load-misses:u                #   60.35% of all LL-cache accesses    (81.82%)

     627.347498426 seconds time elapsed

     602.255726000 seconds user
      20.192128000 seconds sys

Extracting 5694801913 keys...

--- Starting Benchmark ---
Algorithm: Standard C qsort() [Single Threaded]
Sort Time: 628.6752 seconds
Total Performance: 9,058,416 keys-per-second

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 SUPRAPIG_github.com_Sefaria-Project_2024-Jun-01_16833-TXTs.tar 2':

        644,151.70 msec task-clock:u                     #    1.000 CPUs utilized
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
        22,245,410      page-faults:u                    #   34.534 K/sec
 4,332,397,831,502      instructions:u                   #    1.50  insn per cycle              (90.91%)
 2,897,027,251,670      cycles:u                         #    4.497 GHz                         (90.91%)
   822,905,463,934      branches:u                       #    1.278 G/sec                       (90.91%)
    49,541,239,380      branch-misses:u                  #    6.02% of all branches             (81.82%)
                        TopdownL1                 #      1.9 %  tma_backend_bound
                                                  #     98.4 %  tma_bad_speculation
```

```
                                        #     -0.3 %  tma_frontend_bound
                                        #      0.1 %  tma_retiring             (90.91%)
 1,268,500,608,795      L1-dcache-loads:u             #    1.969 G/sec                    (90.91%)
    52,497,326,588      L1-dcache-load-misses:u       #    4.14% of all L1-dcache accesses  (90.91%)
       827,531,392      LLC-loads:u                   #    1.285 M/sec                    (90.91%)
       509,858,962      LLC-load-misses:u             #   61.61% of all LL-cache accesses   (81.82%)

     644.404570882 seconds time elapsed

     619.387145000 seconds user
      20.124794000 seconds sys

Extracting 6126911481 keys...

--- Starting Benchmark ---
Algorithm: Standard C qsort() [Single Threaded]
Sort Time: 860.6229 seconds
Total Performance: 7,119,159 keys-per-second

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 Nobara-43-GNOME-2026-01-26.iso 2':

       878,082.39 msec task-clock:u                     #    1.000 CPUs utilized
                0      context-switches:u              #    0.000 /sec
                0      cpu-migrations:u                #    0.000 /sec
       23,933,294      page-faults:u                   #   27.256 K/sec
 4,978,703,121,441      instructions:u                  #    1.22  insn per cycle            (90.91%)
 4,070,655,649,003      cycles:u                        #    4.636 GHz                       (90.91%)
 1,084,343,675,860      branches:u                      #    1.235 G/sec                     (90.91%)
    98,078,766,609      branch-misses:u                 #    9.04% of all branches           (81.82%)
                        TopdownL1                 #      2.5 %  tma_backend_bound
                                                  #     97.8 %  tma_bad_speculation
                                                  #     -0.5 %  tma_frontend_bound
                                                  #      0.1 %  tma_retiring             (90.91%)
 1,367,276,452,972      L1-dcache-loads:u               #    1.557 G/sec                     (90.91%)
    56,801,443,171      L1-dcache-load-misses:u         #    4.15% of all L1-dcache accesses  (90.91%)
       955,293,720      LLC-loads:u                     #    1.088 M/sec                     (90.91%)
       592,192,679      LLC-load-misses:u               #   61.99% of all LL-cache accesses   (81.82%)

     878.294748079 seconds time elapsed

     849.925523000 seconds user
      22.045097000 seconds sys

Extracting 6126911481 keys...

--- Starting Benchmark ---
Algorithm: Standard C qsort() [Single Threaded]
Sort Time: 941.1377 seconds
Total Performance: 6,510,111 keys-per-second

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 Nobara-43-GNOME-2026-01-26.iso 2':

       958,574.18 msec task-clock:u                     #    1.000 CPUs utilized
                0      context-switches:u              #    0.000 /sec
                0      cpu-migrations:u                #    0.000 /sec
       23,933,302      page-faults:u                   #   24.968 K/sec
 4,670,662,541,916      instructions:u                  #    1.07  insn per cycle            (90.91%)
 4,374,132,625,721      cycles:u                        #    4.563 GHz                       (90.91%)
   887,024,212,481      branches:u                      #  925.358 M/sec                     (90.91%)
   100,577,546,652      branch-misses:u                 #   11.34% of all branches           (81.82%)
                        TopdownL1                 #      2.4 %  tma_backend_bound
                                                  #     97.9 %  tma_bad_speculation
                                                  #     -0.4 %  tma_frontend_bound
                                                  #      0.1 %  tma_retiring             (90.91%)
 1,367,314,942,871      L1-dcache-loads:u               #    1.426 G/sec                     (90.91%)
    56,832,779,980      L1-dcache-load-misses:u         #    4.16% of all L1-dcache accesses  (90.91%)
       927,515,829      LLC-loads:u                     #  967.599 K/sec                     (90.91%)
       568,188,494      LLC-load-misses:u               #   61.26% of all LL-cache accesses   (81.82%)

     958.796468887 seconds time elapsed

     929.460106000 seconds user
      22.423730000 seconds sys
```

```
Extracting 9758687449 keys...

--- Starting Benchmark ---
Algorithm: Standard C qsort() [Single Threaded]
./benchmark28: Killed

 Performance counter stats for './benchmark28 SUPRAPIG_news.ycombinator.com-items-201909_21M-news.ycombinator.com.jsonl 2':

     1,184,889.54 msec task-clock:u                     #    0.997 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
           32,113,221      page-faults:u                    #   27.102 K/sec
 8,136,659,203,902      instructions:u                    #    1.54  insn per cycle              (90.91%)
 5,300,587,386,069      cycles:u                          #    4.473 GHz                         (90.91%)
 1,724,568,030,825      branches:u                        #    1.455 G/sec                       (90.91%)
   106,737,552,222      branch-misses:u                   #    6.19% of all branches             (81.82%)
                        TopdownL1                 #      2.2 %  tma_backend_bound
                                                  #     98.1 %  tma_bad_speculation
                                                  #     -0.4 %  tma_frontend_bound
                                                  #      0.1 %  tma_retiring             (90.91%)
 2,176,715,420,518      L1-dcache-loads:u                 #    1.837 G/sec                       (90.91%)
    90,078,303,839      L1-dcache-load-misses:u           #    4.14% of all L1-dcache accesses   (90.91%)
     1,358,996,489      LLC-loads:u                       #    1.147 M/sec                       (90.91%)
       843,164,809      LLC-load-misses:u                 #   62.04% of all LL-cache accesses    (81.82%)

    1187.971230797 seconds time elapsed

    1148.805797000 seconds user
      30.002362000 seconds sys


Extracting 9758687449 keys...

--- Starting Benchmark ---
Algorithm: Standard C qsort() [Single Threaded]
./benchmark28: Killed

 Performance counter stats for './benchmark28 SUPRAPIG_news.ycombinator.com-items-201909_21M-news.ycombinator.com.jsonl 2':

     1,233,714.87 msec task-clock:u                     #    0.997 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
           32,102,877      page-faults:u                    #   26.021 K/sec
 7,445,969,993,807      instructions:u                    #    1.34  insn per cycle              (90.91%)
 5,575,375,744,923      cycles:u                          #    4.519 GHz                         (90.91%)
 1,407,294,126,792      branches:u                        #    1.141 G/sec                       (90.91%)
   108,315,109,847      branch-misses:u                   #    7.70% of all branches             (81.82%)
                        TopdownL1                 #      2.1 %  tma_backend_bound
                                                  #     98.2 %  tma_bad_speculation
                                                  #     -0.4 %  tma_frontend_bound
                                                  #      0.1 %  tma_retiring             (90.91%)
 2,176,787,236,210      L1-dcache-loads:u                 #    1.764 G/sec                       (90.91%)
    90,097,189,613      L1-dcache-load-misses:u           #    4.14% of all L1-dcache accesses   (90.91%)
     1,333,719,878      LLC-loads:u                       #    1.081 M/sec                       (90.91%)
       811,196,078      LLC-load-misses:u                 #   60.82% of all LL-cache accesses    (81.82%)

    1236.989750378 seconds time elapsed

    1198.151578000 seconds user
      29.279156000 seconds sys

[sanmayce@djuljeto Quicksort_benchmark_2024-Nov-28_update]$
*/

// Quicksort 'Charlatana_UQWORD_AVX2' vs qsort():
// Bottomline first: Quicksort 'Charlatana_UQWORD_AVX2' is 100,548,436 KPS / 9,675,092 KPS = 10.39x faster than qsort().
// Testmachine: Laptop Dell Precision 7560, Core i7-11850H (Tiger Lake-H). 128GB DDR4 3200MHz, OS: Fedora Linux 42
// Testcorpus: Human Genome (sorting all 8 bytes long chuncks - BBs - BuildingBlocks)
// +-------------------------------------+------[ SORTED ]-+--------+-----------------------------------------------+
// | Sort Function, (Compiler)           | Keys-Per-Second |   Time |                  Giga-Instructions-Per-Second |
// +-------------------------------------+-----------------+--------+-----------------------------------------------+
// | Charlatana_UQWORD_AVX512, GCC 15.2.1  | 114,867,502 KPS |  28.8s |   415,877,653,143i/ 28.8s = 14,440,196,289 GIPS |
// | Charlatana_UQWORD_AVX2, GCC 15.2.1    | 100,548,436 KPS |  32.9s |   933,123,132,569i/ 32.9s = 28,362,405,245 GIPS |
// | Charlatana_UQWORD_AVX512, CLANG 20.1.8 |  98,404,826 KPS |  33.6s |   945,379,013,838i/ 33.6s = 28,136,280,173 GIPS |
// | Charlatana_UQWORD_AVX2, CLANG 20.1.8  |  87,842,997 KPS |  37.7s | 1,314,843,354,298i/ 37.7s = 34,876,481,546 GIPS |
// | qsort(),GCC 15.2.1                  |   9,675,092 KPS | 342.4s | 2,642,946,084,482i/342.4s =  7,718,884,592 GIPS |
// | qsort(),CLANG 20.1.8                |   8,881,970 KPS | 373.0s | 2,408,798,523,104i/373.0s =  6,457,904,887 GIPS |
// +-------------------------------------+-----------------+--------+-----------------------------------------------+
```

```
// Note1: Memory footprint is 29,120,748 KB, meaning 32GB RAM are needed.
// Note2: The third Performance statistic, GIPS, is not telling well how fast the compiled-function is, the KPS is the "story teller".
// Note3: The CPU utilization is quite qood (FOR VECTOR CODE), "8.960 CPUs utilized", for the TOP (GCC) result.
// Note4: The testfile is highly redundant (971:1), unique UQWORD-wise, "Unique keys: 3,411,451", that is 3,313,061,624 non-Unique : 3,411,451 Unique.
// Note5: Just wanted to see the single-threaded Charlatana_UQWORD_AVX2 what delivers, "Total Performance: 36,650,159 keys-per-second", i.e. 3x slower it is.

// [sanmayce@djudjeto Quicksort_says_v28v29_FullPackage]$ gcc -O3 -mavx2 -std=gnu11 -o benchmark28_SINGLE.asm benchmark28_SINGLE.c -S
// The Assembler code (for the single-threaded) is this (Insertionsort is embedded/inlined):
/*
        .p2align 4
        .type   charlatana_v28_recursive, @function
charlatana_v28_recursive:
.LFB7297:
        .cfi_startproc
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        pushq   %r15
        pushq   %r14
        pushq   %r13
        pushq   %r12
        pushq   %rbx
        andq    $-32, %rsp
        subq    $4224, %rsp
        .cfi_offset 15, -24
        .cfi_offset 14, -32
        .cfi_offset 13, -40
        .cfi_offset 12, -48
        .cfi_offset 3, -56
        movq    %rdi, 120(%rsp)
        cmpq    %rdx, %rsi
        jge     .L112
        movabsq $-9223372036854775808, %rax
        movq    %rsi, %r10
        movq    %rdx, %r14
        vmovq   %rax, %xmm4
        .p2align 4
        .p2align 3
.L6:
        movq    %r14, %rdx
        movq    120(%rsp), %rbx
        subq    %r10, %rdx
        movq    %rdx, %rcx
        sarq    %rdx
        sarq    $2, %rcx
        addq    %r10, %rdx
        leaq    (%rcx,%r10), %rax
        movq    (%rbx,%rdx,8), %rsi
        movq    %r14, %rdx
        movq    (%rbx,%rax,8), %rax
        subq    %rcx, %rdx
        movq    (%rbx,%rdx,8), %rdx
        movq    %rsi, %rdi
        movq    %rax, %rcx
        xorq    %rsi, %rcx
        xorq    %rdx, %rcx
        cmpq    %rsi, %rax
        cmovbe  %rax, %rdi
        cmpq    %rdx, %rdi
        cmova   %rdx, %rdi
        xorq    %rdi, %rcx
        cmpq    %rsi, %rax
        leaq    (%rbx,%r10,8), %rdi
        cmovb   %rsi, %rax
        movq    %rdi, 16(%rsp)
        cmpq    %rdx, %rax
        cmovb   %rdx, %rax
        xorq    %rax, %rcx
        leaq    1(%r14), %rax
        leaq    (%rbx,%rax,8), %r15
        subq    %r10, %rax
        salq    $3, %rax
        cmpq    $24, %rax
        jle     .L8
        vmovq   %rcx, %xmm6
        movq    %r15, %r8
        vpbroadcastq    %xmm4, %ymm0
```

```
        movq    %rdi, %r12
        vpbroadcastq    %xmm6, %ymm2
        movq    %r14, 24(%rsp)
        movq    %rdi, %r11
        movq    %r15, %r14
        movq    %rcx, 104(%rsp)
        vpxor   %ymm0, %ymm2, %ymm2
        xorl    %ebx, %ebx
        xorl    %r15d, %r15d
        movq    %r10, 96(%rsp)
        movq    %r8, %r13
        jmp     .L11
        .p2align 4,,10
        .p2align 3
.L115:
        vmovdqu (%r11), %ymm1
        vpbroadcastq    %xmm4, %ymm0
        addq    $32, %r11
        movslq  %ebx, %rsi
        vpxor   %ymm0, %ymm1, %ymm0
        vpcmpgtq %ymm0, %ymm2, %ymm3
        vpcmpgtq %ymm2, %ymm0, %ymm0
        vmovmskpd %ymm3, %rdx
        movq    %rdx, %rax
        salq    $5, %rdx
        vmovdqa QWORD_PACK_TABLE(%rdx), %ymm3
        popcntl %eax, %eax
        movq    %rax, %rdx
        salq    $5, %rdx
        vmovdqa MASK_STORE_TABLE(%rdx), %ymm5
        vpermd  %ymm1, %ymm3, %ymm3
        xorl    %edx, %edx
        vpmaskmovq%ymm3, %ymm5, (%r12)
        leaq    (%r12,%rax,8), %r12
        vmovmskpd %ymm0, %rax
        popcntl %eax, %edx
        salq    $5, %rax
        addl    %edx, %ebx
        vmovdqa QWORD_PACK_TABLE(%rax), %ymm0
        movslq  %edx, %rax
        salq    $5, %rax
        vpermd  %ymm1, %ymm0, %ymm1
        vmovdqa MASK_STORE_TABLE(%rax), %ymm0
        movq    %r11, %rax
        subq    %r12, %rax
        vpmaskmovq%ymm1, %ymm0, 2176(%rsp,%rsi,8)
        sarq    $3, %rax
        cmpl    %eax, %r15d
        jg      .L10
        testl   %r15d, %r15d
        jle     .L10
        movq    %r11, 32(%rsp)
        movl    %r15d, %edx
        movq    %r12, %rdi
        leaq    128(%rsp), %rsi
        vmovdqa %ymm2, 64(%rsp)
        salq    $3, %rdx
        movslq  %r15d, %r15
        leaq    (%r12,%r15,8), %r12
        vzeroupper
        call    memcpy
        movq    32(%rsp), %r11
        xorl    %r15d, %r15d
        movabsq $-9223372036854775808, %rax
        vmovdqa 64(%rsp), %ymm2
        vmovq   %rax, %xmm4
.L10:
        movq    %r14, %rax
        subq    %r11, %rax
        cmpq    $24, %rax
        jle     .L114
.L11:
        cmpl    %ebx, %r15d
        jg      .L115
        vmovdqu -32(%r14), %ymm1
        vpbroadcastq    %xmm4, %ymm0
        leaq    -32(%r14), %r9
        movslq  %r15d, %rsi
        vpxor   %ymm0, %ymm1, %ymm0
```

```
        vpcmpgtq  %ymm2, %ymm0, %ymm3
        vpcmpgtq  %ymm0, %ymm2, %ymm0
        vmovmskpd %ymm3, %rdx
        movq      %rdx, %rax
        salq      $5, %rdx
        vmovdqa   QWORD_PACK_TABLE(%rdx), %ymm3
        popcntl   %eax, %eax
        leaq      0(,%rax,8), %rdx
        salq      $5, %rax
        vmovdqa   MASK_STORE_TABLE(%rax), %ymm5
        subq      %rdx, %r13
        vmovmskpd %ymm0, %rax
        xorl      %edx, %edx
        popcntl   %eax, %edx
        vpermd    %ymm1, %ymm3, %ymm3
        salq      $5, %rax
        addl      %edx, %r15d
        vpmaskmovq%ymm3, %ymm5, 0(%r13)
        vmovdqa   QWORD_PACK_TABLE(%rax), %ymm0
        movslq    %edx, %rax
        salq      $5, %rax
        vpermd    %ymm1, %ymm0, %ymm1
        vmovdqa   MASK_STORE_TABLE(%rax), %ymm0
        movq      %r13, %rax
        subq      %r9, %rax
        vpmaskmovq%ymm1, %ymm0, 128(%rsp,%rsi,8)
        sarq      $3, %rax
        cmpl      %eax, %ebx
        jg        .L58
        testl     %ebx, %ebx
        jg        .L116
.L58:
        movq      %r9, %r14
        movq      %r14, %rax
        subq      %r11, %rax
        cmpq      $24, %rax
        jg        .L11
.L114:
        movq      %r13, %r8
        movl      %r15d, %r13d
        movq      %r14, %r15
        movq      104(%rsp), %rcx
        movq      96(%rsp), %r10
        movq      24(%rsp), %r14
        cmpq      %r15, %r11
        jnb       .L15
        .p2align 6
        .p2align 4
        .p2align 3
.L16:
        movq      (%r11), %rax
        xorl      %edx, %edx
        cmpq      %rcx, %rax
        movq      %rax, (%r12)
        setb      %dl
        cmpq      %rax, %rcx
        leaq      (%r12,%rdx,8), %r12
        movslq    %ebx, %rdx
        adcl      $0, %ebx
        addq      $8, %r11
        movq      %rax, 2176(%rsp,%rdx,8)
        cmpq      %r15, %r11
        jb        .L16
.L15:
        testl     %r13d, %r13d
        jle       .L17
        movq      %r10, 112(%rsp)
        movq      %r12, %rdi
        leaq      (%r12,%r13,8), %r12
        leaq      128(%rsp), %rsi
        movq      %rcx, 32(%rsp)
        leaq      0(,%r13,8), %rdx
        movq      %r8, 64(%rsp)
        vzeroupper
        call      memcpy
        movq      112(%rsp), %r10
        movabsq   $-9223372036854775808, %rax
        movq      32(%rsp), %rcx
        movq      64(%rsp), %r8
```

```
        vmovq     %rax, %xmm4
.L17:
        movslq    %ebx, %rax
        salq      $3, %rax
        subq      %rax, %r8
        movq      %r8, %r15
        testl     %ebx, %ebx
        jle       .L18
        movq      %r10, 32(%rsp)
        movl      %ebx, %edx
        movq      %r8, %rdi
        leaq      2176(%rsp), %rsi
        movq      %rcx, 64(%rsp)
        salq      $3, %rdx
        vzeroupper
        call      memcpy
        movq      32(%rsp), %r10
        movabsq   $-9223372036854775808, %rax
        movq      64(%rsp), %rcx
        vmovq     %rax, %xmm4
.L18:
        cmpq      %r15, %r12
        jnb       .L14
        leaq      -1(%r15), %rdx
        movq      %r12, %rax
        subq      %r12, %rdx
        cmpq      $23, %rdx
        jbe       .L22
        shrq      $3, %rdx
        vmovq     %rcx, %xmm7
        addq      $1, %rdx
        vpbroadcastq        %xmm7, %ymm0
        movq      %rdx, %rdi
        shrq      $2, %rdi
        salq      $5, %rdi
        leaq      (%rdi,%r12), %rsi
        andl      $32, %edi
        je        .L23
        leaq      32(%r12), %rax
        vmovdqu   %ymm0, (%r12)
        cmpq      %rax, %rsi
        je        .L106
        .p2align 5
        .p2align 4
        .p2align 3
.L23:
        vmovdqu   %ymm0, (%rax)
        addq      $64, %rax
        vmovdqu   %ymm0, -32(%rax)
        cmpq      %rax, %rsi
        jne       .L23
.L106:
        testb     $3, %dl
        je        .L14
        andq      $-4, %rdx
        leaq      (%r12,%rdx,8), %rax
.L22:
        leaq      8(%rax), %rdx
        movq      %rcx, (%rax)
        cmpq      %r15, %rdx
        jnb       .L14
        leaq      16(%rax), %rdx
        movq      %rcx, 8(%rax)
        cmpq      %r15, %rdx
        jnb       .L14
        movq      %rcx, 16(%rax)
.L14:
        movq      120(%rsp), %rax
        movq      %r15, %rbx
        subq      %rax, %r12
        subq      %rax, %rbx
        sarq      $3, %r12
        sarq      $3, %rbx
        leaq      -1(%r12), %rcx
        movq      %r14, %r12
        movq      %rcx, %r13
        subq      %rbx, %r12
        subq      %r10, %r13
        leaq      1(%r12), %r11
```

```
        leaq    1(%r13), %r9
        cmpq    %r12, %r13
        jge     .L117
        cmpq    $64, %r11
        jle     .L118
        movq    %r9, 112(%rsp)
        movq    %rax, %rdi
        movq    %r14, %rdx
        movq    %rbx, %rsi
        movq    %r10, 32(%rsp)
        movq    %rcx, 64(%rsp)
        vzeroupper
        call    charlatana_v28_recursive
        movq    64(%rsp), %rcx
        movabsq $-9223372036854775808, %rax
        movq    32(%rsp), %r10
        movq    112(%rsp), %r9
        vmovq   %rax, %xmm4
.L29:
        cmpq    $64, %r9
        jle     .L119
.L63:
        movq    %rcx, %r14
.L36:
        cmpq    %r14, %r10
        jl      .L6
.L111:
        vzeroupper
.L112:
        leaq    -40(%rbp), %rsp
        popq    %rbx
        popq    %r12
        popq    %r13
        popq    %r14
        popq    %r15
        popq    %rbp
        .cfi_remember_state
        .cfi_def_cfa 7, 8
        ret
        .p2align 4,,10
        .p2align 3
.L116:
        .cfi_restore_state
        movslq  %ebx, %rax
        movq    %r11, 112(%rsp)
        movl    %ebx, %edx
        leaq    2176(%rsp), %rsi
        salq    $3, %rax
        movq    %r9, 64(%rsp)
        salq    $3, %rdx
        subq    %rax, %r13
        vmovdqa %ymm2, 32(%rsp)
        movq    %r13, %rdi
        vzeroupper
        call    memcpy
        xorl    %ebx, %ebx
        movabsq $-9223372036854775808, %rax
        movq    64(%rsp), %r14
        vmovdqa 32(%rsp), %ymm2
        vmovq   %rax, %xmm4
        movq    112(%rsp), %r11
        jmp     .L10
        .p2align 4,,10
        .p2align 3
.L117:
        cmpq    $64, %r9
        jle     .L120
        movq    %r11, 64(%rsp)
        movq    %rax, %rdi
        movq    %rcx, %rdx
        movq    %r10, %rsi
        vzeroupper
        call    charlatana_v28_recursive
        movq    64(%rsp), %r11
        movabsq $-9223372036854775808, %rax
        vmovq   %rax, %xmm4
.L45:
        cmpq    $64, %r11
        jle     .L121
```

```
.L68:
        movq    %rbx, %r10
        jmp     .L36
        .p2align 4,,10
        .p2align 3
.L118:
        cmpq    $1, %r11
        jle     .L29
        movq    (%r15), %rdi
        movq    %r15, %r14
        leaq    8(%r15), %rdx
        xorl    %ebx, %ebx
        movl    $1, %esi
        movq    %rdi, %r8
        jmp     .L31
        .p2align 5
        .p2align 4,,10
        .p2align 3
.L123:
        leaq    (%r15,%rbx,8), %r11
        addq    $8, %rdx
        cmpq    %rsi, %r12
        je      .L122
.L62:
        addq    $1, %rsi
.L31:
        movq    (%rdx), %rax
        movq    %rdx, %r11
        cmpq    %r8, %rax
        jnb     .L123
        movq    %rax, %r8
        movq    %rsi, %rbx
        addq    $8, %rdx
        cmpq    %rsi, %r12
        jne     .L62
.L122:
        movq    %r8, (%r15)
        leaq    (%r15,%r12,8), %r8
        movq    %rdi, (%r11)
        .p2align 4
        .p2align 3
.L34:
        movq    8(%r14), %rsi
        movq    (%r14), %rdx
        movq    %r14, %rax
        cmpq    %rdx, %rsi
        jnb     .L32
        .p2align 5
        .p2align 4
        .p2align 3
.L33:
        movq    %rdx, 8(%rax)
        movq    -8(%rax), %rdx
        subq    $8, %rax
        cmpq    %rdx, %rsi
        jb      .L33
.L32:
        addq    $8, %r14
        movq    %rsi, 8(%rax)
        cmpq    %r8, %r14
        jne     .L34
        cmpq    $64, %r9
        jg      .L63
.L119:
        cmpq    $1, %r9
        jle     .L111
        movq    16(%rsp), %r11
        xorl    %r9d, %r9d
        movl    $1, %edx
        movq    (%r11), %r10
        movq    %r11, %rsi
        leaq    8(%r11), %rax
        movq    %r10, %rdi
        jmp     .L39
.L125:
        leaq    (%r11,%r9,8), %r8
        addq    $8, %rax
        cmpq    %rdx, %r13
        jle     .L124
```

```
.L65:
        addq    $1, %rdx
.L39:
        movq    (%rax), %rcx
        movq    %rax, %r8
        cmpq    %rdi, %rcx
        jnb     .L125
        movq    %rcx, %rdi
        movq    %rdx, %r9
        addq    $8, %rax
        cmpq    %rdx, %r13
        jg      .L65
.L124:
        movq    16(%rsp), %rax
        movq    %rdi, (%rax)
        xorl    %edi, %edi
        movq    %r10, (%r8)
        .p2align 4
        .p2align 3
.L42:
        movq    8(%rsi), %rcx
        movq    (%rsi), %rdx
        movq    %rsi, %rax
        cmpq    %rdx, %rcx
        jnb     .L40
        .p2align 5
        .p2align 4
        .p2align 3
.L41:
        movq    %rdx, 8(%rax)
        movq    -8(%rax), %rdx
        subq    $8, %rax
        cmpq    %rdx, %rcx
        jb      .L41
.L40:
        addq    $1, %rdi
        movq    %rcx, 8(%rax)
        addq    $8, %rsi
        cmpq    %rdi, %r13
        jg      .L42
        jmp     .L111
        .p2align 4,,10
        .p2align 3
.L120:
        cmpq    $1, %r9
        jle     .L45
        movq    16(%rsp), %r8
        xorl    %r10d, %r10d
        movl    $1, %ecx
        movq    (%r8), %rsi
        movq    %r8, 64(%rsp)
        movq    %r8, %rax
        leaq    8(%r8), %rdx
        movq    %rsi, %rdi
        jmp     .L47
        .p2align 5
        .p2align 4,,10
        .p2align 3
.L127:
        leaq    (%rax,%r10,8), %r8
        addq    $8, %rdx
        cmpq    %rcx, %r13
        je      .L126
.L67:
        addq    $1, %rcx
.L47:
        movq    (%rdx), %r9
        movq    %rdx, %r8
        cmpq    %rdi, %r9
        jnb     .L127
        movq    %r9, %rdi
        movq    %rcx, %r10
        addq    $8, %rdx
        cmpq    %rcx, %r13
        jne     .L67
.L126:
        movq    16(%rsp), %rcx
        movq    64(%rsp), %rax
        movq    %rdi, (%rcx)
```

```
        leaq    (%rcx,%r13,8), %rdi
        movq    %rsi, (%r8)
        .p2align 4
        .p2align 3
.L50:
        movq    8(%rax), %rsi
        movq    (%rax), %rcx
        movq    %rax, %rdx
        cmpq    %rcx, %rsi
        jnb     .L48
        .p2align 5
        .p2align 4
        .p2align 3
.L49:
        movq    %rcx, 8(%rdx)
        movq    -8(%rdx), %rcx
        subq    $8, %rdx
        cmpq    %rcx, %rsi
        jb      .L49
.L48:
        addq    $8, %rax
        movq    %rsi, 8(%rdx)
        cmpq    %rdi, %rax
        jne     .L50
        cmpq    $64, %r11
        jg      .L68
.L121:
        cmpq    $1, %r11
        jle     .L111
        movq    (%r15), %r10
        movq    %r15, %rsi
        leaq    8(%r15), %rax
        xorl    %r9d, %r9d
        movl    $1, %edx
        movq    %r10, %rdi
        jmp     .L53
.L129:
        leaq    (%r15,%r9,8), %r8
        addq    $8, %rax
        cmpq    %rdx, %r12
        jle     .L128
.L70:
        addq    $1, %rdx
.L53:
        movq    (%rax), %rcx
        movq    %rax, %r8
        cmpq    %rdi, %rcx
        jnb     .L129
        movq    %rcx, %rdi
        movq    %rdx, %r9
        addq    $8, %rax
        cmpq    %rdx, %r12
        jg      .L70
.L128:
        movq    %rdi, (%r15)
        xorl    %edi, %edi
        movq    %r10, (%r8)
        .p2align 4
        .p2align 3
.L56:
        movq    8(%rsi), %rcx
        movq    (%rsi), %rdx
        movq    %rsi, %rax
        cmpq    %rdx, %rcx
        jnb     .L54
        .p2align 5
        .p2align 4
        .p2align 3
.L55:
        movq    %rdx, 8(%rax)
        movq    -8(%rax), %rdx
        subq    $8, %rax
        cmpq    %rdx, %rcx
        jb      .L55
.L54:
        addq    $1, %rdi
        movq    %rcx, 8(%rax)
        addq    $8, %rsi
        cmpq    %rdi, %r12
```

```
        jg      .L56
        jmp     .L111
.L8:
        movq    %rdi, %r12
        cmpq    %r15, 16(%rsp)
        jnb     .L14
        movq    %rdi, %r11
        movq    %r15, %r8
        xorl    %ebx, %ebx
        xorl    %r13d, %r13d
        jmp     .L16
        .cfi_endproc
.LFE7297:
        .size   charlatana_v28_recursive, .-charlatana_v28_recursive

        .align 32
        .type   MASK_STORE_TABLE, @object
        .size   MASK_STORE_TABLE, 160
MASK_STORE_TABLE:
        .quad   0
        .quad   0
        .quad   0
        .quad   0
        .quad   -1
        .quad   0
        .quad   0
        .quad   0
        .quad   -1
        .quad   -1
        .quad   0
        .quad   0
        .quad   -1
        .quad   -1
        .quad   -1
        .quad   0
        .quad   -1
        .quad   -1
        .quad   -1
        .quad   -1
        .align 32
        .type   QWORD_PACK_TABLE, @object
        .size   QWORD_PACK_TABLE, 512
QWORD_PACK_TABLE:
        .long   0
        .long   1
        .long   2
        .long   3
        .long   4
        .long   5
        .long   6
        .long   7
        .long   0
        .long   1
        .long   2
        .long   3
        .long   4
        .long   5
        .long   6
        .long   7
        .long   2
        .long   3
        .long   0
        .long   1
        .long   4
        .long   5
        .long   6
        .long   7
        .long   0
        .long   1
        .long   2
        .long   3
        .long   4
        .long   5
        .long   6
        .long   7
        .long   4
        .long   5
        .long   0
        .long   1
```

```
        .long   2
        .long   3
        .long   6
        .long   7
        .long   0
        .long   1
        .long   4
        .long   5
        .long   2
        .long   3
        .long   6
        .long   7
        .long   2
        .long   3
        .long   4
        .long   5
        .long   0
        .long   1
        .long   6
        .long   7
        .long   0
        .long   1
        .long   2
        .long   3
        .long   4
        .long   5
        .long   6
        .long   7
        .long   6
        .long   7
        .long   0
        .long   1
        .long   2
        .long   3
        .long   4
        .long   5
        .long   0
        .long   1
        .long   6
        .long   7
        .long   2
        .long   3
        .long   4
        .long   5
        .long   2
        .long   3
        .long   6
        .long   7
        .long   0
        .long   1
        .long   4
        .long   5
        .long   0
        .long   1
        .long   2
        .long   3
        .long   6
        .long   7
        .long   4
        .long   5
        .long   4
        .long   5
        .long   6
        .long   7
        .long   0
        .long   1
        .long   2
        .long   3
        .long   0
        .long   1
        .long   4
        .long   5
        .long   6
        .long   7
        .long   2
        .long   3
        .long   2
        .long   3
        .long   4
```

```
        .long   5
        .long   6
        .long   7
        .long   0
        .long   1
        .long   0
        .long   1
        .long   2
        .long   3
        .long   4
        .long   5
        .long   6
        .long   7
*/

/*
[sanmayce@djudjeto Quicksort_benchmark_2024-Nov-28_update_2026-Mar-18]$ sh bench28.sh
20.1.8
15.2.1

Extracting 3313061624 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 32.9499 seconds
Total Performance: 100,548,436 keys-per-second
Unique keys: 3,411,451

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 www.ncbi.nlm.nih.gov_genome_guide_human_GRCh38_latest_genomic.fna 1':

        390,203.02 msec task-clock:u                     #    8.960 CPUs utilized
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
         6,471,342      page-faults:u                    #   16.585 K/sec
   933,123,132,569      instructions:u                   #    0.82  insn per cycle              (90.90%)
 1,137,294,040,482      cycles:u                         #    2.915 GHz                         (90.91%)
   117,698,509,837      branches:u                       #  301.634 M/sec                       (90.91%)
     6,089,659,651      branch-misses:u                  #    5.17% of all branches             (81.82%)
                        TopdownL1                #      3.0 %  tma_backend_bound
                                                 #     93.2 %  tma_bad_speculation
                                                 #     -0.1 %  tma_frontend_bound
                                                 #      3.9 %  tma_retiring             (90.91%)
   144,832,727,838      L1-dcache-loads:u                #  371.173 M/sec                       (90.91%)
    10,634,282,776      L1-dcache-load-misses:u          #    7.34% of all L1-dcache accesses   (90.91%)
       209,767,067      LLC-loads:u                      #  537.584 K/sec                       (90.91%)
       200,605,979      LLC-load-misses:u                #   95.63% of all LL-cache accesses    (81.81%)

      43.548402808 seconds time elapsed

     369.553758000 seconds user
       6.366808000 seconds sys

Extracting 3313061624 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 37.7157 seconds
Total Performance: 87,842,997 keys-per-second
Unique keys: 3,411,451

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 www.ncbi.nlm.nih.gov_genome_guide_human_GRCh38_latest_genomic.fna 1':

        567,610.13 msec task-clock:u                     #   11.785 CPUs utilized
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
         6,471,567      page-faults:u                    #   11.401 K/sec
 1,314,843,354,298      instructions:u                   #    0.80  insn per cycle              (90.91%)
 1,640,261,908,377      cycles:u                         #    2.890 GHz                         (90.91%)
   208,258,520,895      branches:u                       #  366.904 M/sec                       (90.91%)
     8,226,881,845      branch-misses:u                  #    3.95% of all branches             (81.82%)
                        TopdownL1                #      7.7 %  tma_backend_bound
```

```
                                          #     89.9 %  tma_bad_speculation
                                          #      0.0 %  tma_frontend_bound
                                          #      2.4 %  tma_retiring             (90.91%)
   246,429,661,289      L1-dcache-loads:u               #  434.153 M/sec                       (90.91%)
    13,385,049,064      L1-dcache-load-misses:u         #    5.43% of all L1-dcache accesses   (90.91%)
     1,739,365,399      LLC-loads:u                     #    3.064 M/sec                       (90.91%)
       249,325,625      LLC-load-misses:u               #   14.33% of all LL-cache accesses    (81.82%)

      48.165520187 seconds time elapsed

     541.565271000 seconds user
       8.384866000 seconds sys

Extracting 3313061624 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v28 (Branchless OpenMP + AVX2 In-Place)
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 37.1803 seconds
Total Performance: 89,108,023 keys-per-second
Unique keys: 3,411,451

--- Verification ---
Array properly sorted: YES
        Command being timed: "./benchmark28 www.ncbi.nlm.nih.gov_genome_guide_human_GRCh38_latest_genomic.fna 1"
        User time (seconds): 546.98
        System time (seconds): 8.07
        Percent of CPU this job got: 1174%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:47.26
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 29120748
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 7280436
        Voluntary context switches: 78
        Involuntary context switches: 11525
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0

Extracting 3313061624 keys...

--- Starting Benchmark ---
Algorithm: Standard C qsort() [Single Threaded]
Sort Time: 342.4320 seconds
Total Performance: 9,675,092 keys-per-second
Unique keys: 3,411,451

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 www.ncbi.nlm.nih.gov_genome_guide_human_GRCh38_latest_genomic.fna 2':

       352,950.26 msec task-clock:u                     #    1.000 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
       12,941,726      page-faults:u                    #   36.667 K/sec
 2,642,946,084,482      instructions:u                   #    1.63  insn per cycle              (90.91%)
 1,617,911,635,969      cycles:u                         #    4.584 GHz                         (90.91%)
   558,879,975,061      branches:u                       #    1.583 G/sec                       (90.91%)
    30,099,998,784      branch-misses:u                  #    5.39% of all branches             (81.82%)
                        TopdownL1                 #      2.1 %  tma_backend_bound
                                                  #     98.0 %  tma_bad_speculation
                                                  #     -0.3 %  tma_frontend_bound
                                                  #      0.2 %  tma_retiring             (90.91%)
   705,872,542,626      L1-dcache-loads:u                #    2.000 G/sec                       (90.91%)
    29,669,347,089      L1-dcache-load-misses:u          #    4.20% of all L1-dcache accesses   (90.91%)
       629,263,511      LLC-loads:u                      #    1.783 M/sec                       (90.91%)
       430,778,208      LLC-load-misses:u                #   68.46% of all LL-cache accesses    (81.82%)

     353.036213165 seconds time elapsed
```

```
     338.661765000 seconds user
      11.766211000 seconds sys

Extracting 3313061624 keys...

--- Starting Benchmark ---
Algorithm: Standard C qsort() [Single Threaded]
Sort Time: 373.0097 seconds
Total Performance: 8,881,970 keys-per-second
Unique keys: 3,411,451

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark28 www.ncbi.nlm.nih.gov_genome_guide_human_GRCh38_latest_genomic.fna 2':

       383,391.29 msec task-clock:u                     #    1.000 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
       12,941,739      page-faults:u                    #   33.756 K/sec
2,408,798,523,104      instructions:u                    #    1.42  insn per cycle              (90.91%)
1,699,651,544,982      cycles:u                         #    4.433 GHz                         (90.91%)
  457,151,900,824      branches:u                       #    1.192 G/sec                       (90.91%)
   30,991,379,910      branch-misses:u                  #    6.78% of all branches             (81.82%)
                       TopdownL1                 #     2.0 %  tma_backend_bound
                                                 #    98.1 %  tma_bad_speculation
                                                 #    -0.3 %  tma_frontend_bound
                                                 #     0.2 %  tma_retiring             (90.91%)
  705,816,233,788      L1-dcache-loads:u                #    1.841 G/sec                       (90.91%)
   29,690,516,507      L1-dcache-load-misses:u          #    4.21% of all L1-dcache accesses   (90.91%)
      679,215,631      LLC-loads:u                      #    1.772 M/sec                       (90.91%)
      488,482,310      LLC-load-misses:u                #   71.92% of all LL-cache accesses    (81.82%)

     383.479187688 seconds time elapsed

     368.879978000 seconds user
      11.699436000 seconds sys

[sanmayce@djudjeto Quicksort_benchmark_2024-Nov-28_update_2026-Mar-18]$ fastfetch
            .',;::::;,'.                 sanmayce@djudjeto
        .';:cccccccccccc:;,.             -----------------
     .;cccccccccccccccccccccc;.          OS: Fedora Linux 42 (MATE-Compiz) x86_64
   .:cccccccccccccccccccccccccc:.        Host: Precision 7560
  .;ccccccccccccc;.:dddl:.;ccccccc;.     Kernel: Linux 6.16.4-200.fc42.x86_64
 .:ccccccccccccc;OWMKOOXMWd;ccccccc:.    Uptime: 21 days, 16 hours, 5 mins
.:ccccccccccccc;KMMc;cc;xMMc;ccccccc:.   Packages: 5234 (rpm), 13 (snap)
,cccccccccccccc;MMM.;cc;;WW:;cccccccc,   Shell: bash 5.2.37
:cccccccccccccc;MMM.;cccccccccccccccc:   Display (LGD05F2): 1920x1080 @ 60 Hz in 16" [Built-in]
:ccccccc;oxOOOo;MMM000k.;ccccccccccc:    DE: Mate 1.28.2
cccccc;0MMKxdd:;MMMkddc.;ccccccccccc;    WM: Marco (X11)
ccccc;XMO';cccc;MMM.;ccccccccccccccc'    WM Theme: Spidey-Left
ccccc;MMo;ccccc;MMW.;ccccccccccccccc;    Theme: Adwaita-dark [GTK2/3/4]
ccccc;0MNc.ccc.xMMd;ccccccccccccccc;     Icons: mate [GTK2/3/4]
cccccc;dNMWXXXWM0:;ccccccccccccc:,       Font: MxPlus ToshibaTxL2 8x16 (16pt) [GTK2/3/4]
ccccccc;.:odl:.;ccccccccccccc:,.         Cursor: mate (24px)
cccccccccccccccccccccccccccccc:'.        Terminal: mate-terminal 1.28.1
:ccccccccccccccccccccccc:;,..            Terminal Font: MxPlus ToshibaTxL2 8x16 (16pt)
 ':ccccccccccccccc::;,.                  CPU: 11th Gen Intel(R) Core(TM) i7-11850H (16) @ 4.80 GHz
                                         GPU: Intel UHD Graphics @ 1.45 GHz [Integrated]
                                         Memory: 2.94 GiB / 125.00 GiB (2%)
                                         Swap: Disabled
                                         Disk (/): 213.76 GiB / 1.79 TiB (12%) - ext4
                                         Disk (/run/media/sanmayce/A400-480_Master): 285.03 GiB / 447.13 GiB (64%) - fuseblk
                                         Disk (/run/media/sanmayce/A400-960GB_Movies): 867.97 GiB / 894.25 GiB (97%) - fuseblk
                                         Disk (/run/media/sanmayce/Lexar_4TB): 3.46 TiB / 3.67 TiB (94%) - ext4
                                         Disk (/run/media/sanmayce/Torbalan_MP600): 1.71 TiB / 1.82 TiB (94%) - fuseblk
                                         Disk (/run/media/sanmayce/sx): 318.21 GiB / 477.50 GiB (67%) - fuseblk
                                         Battery (DELL CR72X21): 68% [AC Connected]
                                         Locale: en_US.UTF-8
*/

// The 512 beast:
/*
[sanmayce@djudjeto Quicksort_benchmark_2024-Nov-28_update_2026-Mar-18]$ gcc -O3 -mavx512f -fopenmp -std=gnu11 -o benchmark29 benchmark29.c
[sanmayce@djudjeto Quicksort_benchmark_2024-Nov-28_update_2026-Mar-18]$ perf stat -d ./benchmark29
www.ncbi.nlm.nih.gov_genome_guide_human_GRCh38_latest_genomic.fna 1
Extracting 3313061624 keys...
```

```
--- Starting Benchmark ---
Algorithm: Charlatana_v29
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 28.8425 seconds
Total Performance: 114,867,502 keys-per-second
Unique keys: 3,411,451

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark29 www.ncbi.nlm.nih.gov_genome_guide_human_GRCh38_latest_genomic.fna 1':

        317,749.15 msec task-clock:u                     #    8.077 CPUs utilized
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
         6,471,337      page-faults:u                    #   20.366 K/sec
   415,877,653,143      instructions:u                   #    0.38  insn per cycle              (90.91%)
 1,084,714,946,647      cycles:u                         #    3.414 GHz                         (90.90%)
    68,292,683,988      branches:u                       #  214.926 M/sec                       (90.91%)
     2,506,529,767      branch-misses:u                  #    3.67% of all branches             (81.82%)
                        TopdownL1                #     14.3 %  tma_backend_bound
                                                 #     84.7 %  tma_bad_speculation
                                                 #     -0.2 %  tma_frontend_bound
                                                 #      1.1 %  tma_retiring             (90.91%)
    58,696,843,153      L1-dcache-loads:u                #  184.727 M/sec                       (90.91%)
    10,900,937,425      L1-dcache-load-misses:u          #   18.57% of all L1-dcache accesses   (90.91%)
       500,860,752      LLC-loads:u                      #    1.576 M/sec                       (90.91%)
       485,751,397      LLC-load-misses:u                #   96.98% of all LL-cache accesses    (81.82%)

      39.339132921 seconds time elapsed

     302.794319000 seconds user
       6.382763000 seconds sys


[sanmayce@djudjeto Quicksort_benchmark_2024-Nov-28_update_2026-Mar-18]$ clang -O3 -mavx512f -fopenmp -std=gnu11 -o benchmark29 benchmark29.c
[sanmayce@djudjeto Quicksort_benchmark_2024-Nov-28_update_2026-Mar-18]$ perf stat -d ./benchmark29
www.ncbi.nlm.nih.gov_genome_guide_human_GRCh38_latest_genomic.fna 1
Extracting 3313061624 keys...

--- Starting Benchmark ---
Algorithm: Charlatana_v29
OpenMP Engine: Utilizing up to 16 Hardware Threads
Sort Time: 33.6677 seconds
Total Performance: 98,404,826 keys-per-second
Unique keys: 3,411,451

--- Verification ---
Array properly sorted: YES

 Performance counter stats for './benchmark29 www.ncbi.nlm.nih.gov_genome_guide_human_GRCh38_latest_genomic.fna 1':

        514,764.86 msec task-clock:u                     #   11.222 CPUs utilized
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
         6,471,556      page-faults:u                    #   12.572 K/sec
   945,379,013,838      instructions:u                   #    0.59  insn per cycle              (90.91%)
 1,592,955,931,194      cycles:u                         #    3.095 GHz                         (90.91%)
   180,069,044,361      branches:u                       #  349.808 M/sec                       (90.91%)
     5,278,231,770      branch-misses:u                  #    2.93% of all branches             (81.82%)
                        TopdownL1                #      8.1 %  tma_backend_bound
                                                 #     90.1 %  tma_bad_speculation
                                                 #      0.1 %  tma_frontend_bound
                                                 #      1.7 %  tma_retiring             (90.91%)
   191,185,019,866      L1-dcache-loads:u                #  371.403 M/sec                       (90.91%)
    13,942,845,674      L1-dcache-load-misses:u          #    7.29% of all L1-dcache accesses   (90.91%)
     2,030,920,450      LLC-loads:u                      #    3.945 M/sec                       (90.91%)
       368,229,724      LLC-load-misses:u                #   18.13% of all LL-cache accesses    (81.82%)

      45.872662761 seconds time elapsed

     492.585142000 seconds user
       8.900211000 seconds sys


[sanmayce@djudjeto Quicksort_benchmark_2024-Nov-28_update_2026-Mar-18]$
*/
```